databricks ingestion / 0.ingest_all_files

(https://databricks.com)

```
dbutils.notebook.help()
```

The notebook module.

exit(value: String): void -> This method lets you exit a notebook with a value
run(path: String, timeoutSeconds: int, arguments: Map): String -> This method runs a notebook and returns its exit value

```
v_result = dbutils.notebook.run("1.ingest_circuits_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
```

```
%sql
select race_id, count(1)
from f1_processed.results
group by race_id
order by race_id desc;
```

Table

	race_id	count(1)
1	1052	20

1 row

```
v_result
```

```
'Success'
```

```
v_result = dbutils.notebook.run("2.ingest_races_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
v_result
```

```
'Success'
```

```
1  v_result = dbutils.notebook.run("3.ingest_constructors_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"}
2  v_result
3
4
```

```
> Py4JJavaError: An error occurred while calling o433.run.
: com.databricks.WorkflowException: com.databricks.NotebookExecutionException: FAILED: Unable to access the notebook "/Formula1/in
gestion/3.ingest_constructors_file". Either it does not exist, or the identity used to run this job, chetan chandra (chetanchandr…
File <command-646663711026979>, line 1
----> 1 v_result = dbutils.notebook.run("3.ingest_constructors_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-1
8"})
      2 v_result
```

```
File /databricks/spark/python/lib/py4j-0.10.9.7-src.zip/py4j/protocol.py:326, in get_return_value(answer, gateway_client, target_id, name)
    324 value = OUTPUT_CONVERTER[type](answer[2:], gateway_client)
    325 if answer[1] == REFERENCE_TYPE:
--> 326     raise Py4JJavaError(
    327         "An error occurred while calling {0}{1}{2}.\n".
    328         format(target_id, ".", name), value)
    329 else:
    330     raise Py4JError(
    331         "An error occurred while calling {0}{1}{2}. Trace:\n{3}\n".
    332         format(target_id, ".", name, value))
```

```
1  v_result = dbutils.notebook.run("4.ingest_drivers_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
2  v_result
```

```
> Py4JJavaError: An error occurred while calling o416.run.
: com.databricks.WorkflowException: com.databricks.NotebookExecutionException: FAILED: Unable to access the notebook "/Formula1/ingestion/4.ingest_drivers_file". Either it does not exist, or the identity used to run this job, chetan chandra (chetanchandra12@o…
File <command-646663711026980>, line 1
----> 1 v_result = dbutils.notebook.run("4.ingest_drivers_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
      2 v_result

File /databricks/spark/python/lib/py4j-0.10.9.7-src.zip/py4j/protocol.py:326, in get_return_value(answer, gateway_client, target_id, name)
    324 value = OUTPUT_CONVERTER[type](answer[2:], gateway_client)
    325 if answer[1] == REFERENCE_TYPE:
--> 326     raise Py4JJavaError(
    327         "An error occurred while calling {0}{1}{2}.\n".
    328         format(target_id, ".", name), value)
    329 else:
    330     raise Py4JError(
    331         "An error occurred while calling {0}{1}{2}. Trace:\n{3}\n".
    332         format(target_id, ".", name, value))
```

```
v_result = dbutils.notebook.run("5.ingest_results_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
v_result
```

```
'Success'
```

```
v_result = dbutils.notebook.run("6.ingest_pit_stops_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
v_result
```

```
'Success'
```

```
v_result = dbutils.notebook.run("7.ingest_lap_times_file", 0, {"p_data_source": "Ergast API", "p_file_date": "2021-04-18"})
v_result
```

```
'Success'
```

```
'Success'
```

databricks Set-up / 1.1.access_adls_using_cluster_scoped_credentials

(https://databricks.com)

```
formula1dl_account_key = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1dl-account-key')
```

```
spark.conf.set(
    "fs.azure.account.key.formula1dl1204.dfs.core.windows.net",
    formula1dl_account_key)
```

```
display(dbutils.fs.ls("abfss://demo@formula1dl1204.dfs.core.windows.net"))
```

Table

	path	name	size	modificationTime
1	abfss://demo@formula1dl1204.dfs.core.windows.net/circuits.c...	circuits.csv	10044	1721470200000

1 row

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7
1	circuitId	circuitRef	name	location	country	lat	lng	alt
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18
4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109
6	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.405	130
7	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
8	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228	13
9	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361	228
10	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153
11	10	hockenheimri...	Hockenheimring	Hockenheim	Germany	49.3278	8.56583	103
12	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.2486	264
13	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667	4
14	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139	401

78 rows

databricks Set-up / 1.access_adls_using_access_keys

(https://databricks.com)

```
formula1dl_account_key = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1dl-account-key')
```

```
spark.conf.set(
    "fs.azure.account.key.formula1dl1204.dfs.core.windows.net",
    formula1dl_account_key)
```

```
display(dbutils.fs.ls("abfss://demo@formula1dl1204.dfs.core.windows.net"))
```

Table

	path	name	size	modificationTime
1	abfss://demo@formula1dl1204.dfs.core.windows.net/circuits.c...	circuits.csv	10044	1721470200000

1 row

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7
1	circuitId	circuitRef	name	location	country	lat	lng	alt
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18
4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109
6	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.405	130
7	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
8	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228	13
9	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361	228
10	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153
11	10	hockenheimri...	Hockenheimring	Hockenheim	Germany	49.3278	8.56583	103
12	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.2486	264
13	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667	4
14	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139	401

78 rows

```
create database if not exists f1_raw;
```

OK

Create tables for CSV files

Create circuits table

```
DROP TABLE IF EXISTS f1_raw.circuits;
CREATE TABLE IF NOT EXISTS f1_raw.circuits
(circuitId INT,
circuitRef STRING,
name STRING,
location STRING,
country STRING,
lat DOUBLE,
lng DOUBLE,
alt INT,
url STRING
)
USING csv
OPTIONS (path "/mnt/formula1dl1204/raw/circuits.csv", header true)
```

OK

```
SELECT * FROM f1_raw.circuits;
```

Table

	circuitId	circuitRef	name	location	country	lat	lng
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.96
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.73
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.510
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.2611
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.40
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.4205
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.522
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.1636
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.0169
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	8.5658
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.248
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.33166
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.9713
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.2811

77 rows

Create races table

```
DROP TABLE IF EXISTS f1_raw.races;
CREATE TABLE IF NOT EXISTS f1_raw.races
(raceId INT,
year INT,
round INT,
circuitId INT,
name STRING,
date DATE,
time STRING,
url STRING)
USING csv
OPTIONS (path "/mnt/formula1dl1204/raw/races.csv", header true)
```

OK

```
SELECT * FROM f1_raw.races;
```

Table

raceId	year	round	circuitId	name	date	time	url
26	2008	9	9	British Grand Prix	2008-07-06	12:00:00	http://en.wikipedia.org
27	2008	10	10	German Grand Prix	2008-07-20	12:00:00	http://en.wikipedia.org
28	2008	11	11	Hungarian Grand Prix	2008-08-03	12:00:00	http://en.wikipedia.org
29	2008	12	12	European Grand Prix	2008-08-24	12:00:00	http://en.wikipedia.org
30	2008	13	13	Belgian Grand Prix	2008-09-07	12:00:00	http://en.wikipedia.org
31	2008	14	14	Italian Grand Prix	2008-09-14	12:00:00	http://en.wikipedia.org
32	2008	15	15	Singapore Grand Prix	2008-09-28	12:00:00	http://en.wikipedia.org
33	2008	16	16	Japanese Grand Prix	2008-10-12	04:30:00	http://en.wikipedia.org
34	2008	17	17	Chinese Grand Prix	2008-10-19	07:00:00	http://en.wikipedia.org
35	2008	18	18	Brazilian Grand Prix	2008-11-02	17:00:00	http://en.wikipedia.org
36	2007	1	1	Australian Grand Prix	2007-03-18	03:00:00	http://en.wikipedia.org
37	2007	2	2	Malaysian Grand Prix	2007-04-08	07:00:00	http://en.wikipedia.org
38	2007	3	3	Bahrain Grand Prix	2007-04-15	11:30:00	http://en.wikipedia.org
39	2007	4	4	Spanish Grand Prix	2007-05-13	12:00:00	http://en.wikipedia.org
40	2007	5	6	Monaco Grand Prix	2007-05-27	12:00:00	http://en.wikipedia.org
41	2007	6	7	Canadian Grand Prix	2007-06-10	17:00:00	http://en.wikipedia.org

1,058 rows

Create tables for JSON files

Create constructors table

- Single Line JSON
- Simple structure

```
DROP TABLE IF EXISTS f1_raw.constructors;
CREATE TABLE IF NOT EXISTS f1_raw.constructors
(
constructorId INT,
constructorRef STRING,
name STRING,
nationality STRING,
url STRING)
USING json
OPTIONS(path "/mnt/formula1dl1204/raw/constructors.json")
```

OK

```
SELECT * FROM f1_raw.constructors;
```

Table

	constructorId	constructorRef	name	nationality	url
1	1	mclaren	McLaren	British	http://en.wikipedia.org/wiki/McLaren
2	2	bmw_sauber	BMW Sauber	German	http://en.wikipedia.org/wiki/BMW_Sauber
3	3	williams	Williams	British	http://en.wikipedia.org/wiki/Williams_Grand_Prix_Engineering
4	4	renault	Renault	French	http://en.wikipedia.org/wiki/Renault_in_Formula_One
5	5	toro_rosso	Toro Rosso	Italian	http://en.wikipedia.org/wiki/Scuderia_Toro_Rosso
6	6	ferrari	Ferrari	Italian	http://en.wikipedia.org/wiki/Scuderia_Ferrari
7	7	toyota	Toyota	Japanese	http://en.wikipedia.org/wiki/Toyota_Racing
8	8	super_aguri	Super Aguri	Japanese	http://en.wikipedia.org/wiki/Super_Aguri_F1
9	9	red_bull	Red Bull	Austrian	http://en.wikipedia.org/wiki/Red_Bull_Racing
10	10	force_india	Force India	Indian	http://en.wikipedia.org/wiki/Racing_Point_Force_India
11	11	honda	Honda	Japanese	http://en.wikipedia.org/wiki/Honda_Racing_F1
12	12	spyker	Spyker	Dutch	http://en.wikipedia.org/wiki/Spyker_F1
13	13	mf1	MF1	Russian	http://en.wikipedia.org/wiki/Midland_F1_Racing
14	14	spyker_mf1	Spyker MF1	Dutch	http://en.wikipedia.org/wiki/Midland_F1_Racing

211 rows

Create drivers table

- Single Line JSON
- Complex structure

```
DROP TABLE IF EXISTS f1_raw.drivers;
CREATE TABLE IF NOT EXISTS f1_raw.drivers
(
driverId INT,
driverRef STRING,
number INT,
code STRING,
name STRUCT<forename: STRING, surname: STRING>,
dob DATE,
nationality STRING,
url STRING)
USING json
OPTIONS (path "/mnt/formula1dl1204/raw/drivers.json")
```

OK

Create results table

- Single Line JSON
- Simple structure

```
DROP TABLE IF EXISTS f1_raw.results;
CREATE TABLE IF NOT EXISTS f1_raw.results(
resultId INT,
raceId INT,
driverId INT,
constructorId INT,
number INT,grid INT,
position INT,
positionText STRING,
positionOrder INT,
points INT,
laps INT,
time STRING,
milliseconds INT,
fastestLap INT,
rank INT,
fastestLapTime STRING,
fastestLapSpeed FLOAT,
statusId STRING)
USING json
OPTIONS(path "/mnt/formula1dl1204/raw/results.json")
```

OK

```
SELECT * FROM f1_raw.results
```

Table

	resultId	raceId	driverId	constructorId	number	grid	position	positionTe
80	80	21	3	3	7	15	null	
81	81	21	4	4	5	2	null	R
82	82	21	22	11	17	11	null	R
83	83	21	5	1	23	6	null	R
84	84	21	19	8	19	21	null	R
85	85	21	7	5	14	16	null	R
86	86	21	12	4	6	10	null	R
87	87	21	16	10	20	20	null	R
88	88	21	20	5	15	18	null	R
89	89	22	13	6	2	1	1	1
90	90	22	1	1	22	3	2	2
91	91	22	8	6	1	4	3	3
92	92	22	9	2	4	5	4	4
93	93	22	2	2	3	9	5	5
94	94	22	4	4	5	7	6	6

10,000+ rows | Truncated data due to row limit

Create pit stops table

- Multi Line JSON
- Simple structure

```
DROP TABLE IF EXISTS f1_raw.pit_stops;
CREATE TABLE IF NOT EXISTS f1_raw.pit_stops(
driverId INT,
duration STRING,
lap INT,
milliseconds INT,
raceId INT,
stop INT,
time STRING)
USING json
OPTIONS(path "/mnt/formula1dl1204/raw/pit_stops.json", multiLine true)
```

OK

```
SELECT * FROM f1_raw.pit_stops;
```

Table

	driverId	duration	lap	milliseconds	raceId	stop	time
1	153	26.898	1	26898	841	1	17:05:23
2	30	25.021	1	25021	841	1	17:05:52
3	17	23.426	11	23426	841	1	17:20:48
4	4	23.251	12	23251	841	1	17:22:34
5	13	23.842	13	23842	841	1	17:24:10
6	22	23.643	13	23643	841	1	17:24:29
7	20	22.603	14	22603	841	1	17:25:17
8	814	24.863	14	24863	841	1	17:26:03
9	816	25.259	14	25259	841	1	17:26:50
10	67	25.342	15	25342	841	1	17:27:34
11	2	22.994	15	22994	841	1	17:27:41
12	1	23.227	16	23227	841	1	17:28:24
13	808	24.535	16	24535	841	1	17:28:39
14	3	23.716	16	23716	841	1	17:29:00
15	155	24.064	16	24064	841	1	17:29:06

8,030 rows

Create tables for list of files

Create Lap Times Table

- CSV file
- Multiple files

```
DROP TABLE IF EXISTS f1_raw.lap_times;
CREATE TABLE IF NOT EXISTS f1_raw.lap_times(
raceId INT,
driverId INT,
lap INT,
position INT,
time STRING,
milliseconds INT
)
USING csv
OPTIONS (path "/mnt/formula1dl1204/raw/lap_times")
```

OK

```
SELECT * FROM f1_raw.lap_times
```

Table

	raceId	driverId	lap	position	time	milliseconds
26	841	20	26	1	1:31.328	91328
27	841	20	27	1	1:31.568	91568
28	841	20	28	1	1:31.113	91113
29	841	20	29	1	1:31.339	91339
30	841	20	30	1	1:31.054	91054
31	841	20	31	1	1:31.707	91707
32	841	20	32	1	1:31.611	91611
33	841	20	33	1	1:31.406	91406
34	841	20	34	1	1:31.871	91871
35	841	20	35	1	1:32.597	92597
36	841	20	36	1	1:53.737	113737
37	841	20	37	1	1:39.321	99321
38	841	20	38	1	1:33.632	93632
39	841	20	39	1	1:31.005	91005
40	841	20	40	1	1:30.530	90530
41	841	20	41	1	1:30.140	90140
42	841	20	42	1	1:30.419	90419

10,000+ rows | Truncated data due to row limit

Create Qualifying Table

- JSON file
- MultiLine JSON
- Multiple files

```
DROP TABLE IF EXISTS f1_raw.qualifying;
CREATE TABLE IF NOT EXISTS f1_raw.qualifying(
constructorId INT,
driverId INT,
number INT,
position INT,
q1 STRING,
q2 STRING,
q3 STRING,
qualifyId INT,
raceId INT)
USING json
OPTIONS (path "/mnt/formula1dl1204/raw/qualifying", multiLine true)
```

OK

```
SELECT * FROM f1_raw.qualifying
```

Table

	constructorId	driverId	number	position	q1	q2	q3	qualifyId
1	1	1	22	1	1:26.572	1:25.187	1:26.714	1
2	2	9	4	2	1:26.103	1:25.315	1:26.869	2
3	1	5	23	3	1:25.664	1:25.452	1:27.079	3
4	6	13	2	4	1:25.994	1:25.691	1:27.178	4
5	2	2	3	5	1:25.960	1:25.518	1:27.236	5

databricks demo / 1.filter_demo

(https://databricks.com)

```
%run "../includes/configuration"
```

```
races_df = spark.read.parquet(f"{processed_folder_path}/races")
```

```
display(races_df)
```

Table

	race_id	race_year	round	circuit_id	name	ingestion_date	race_timesta
21	21	2008	4	4	Spanish Grand Prix	2024-07-21T09:58:36.897+00:...	2008-04-27T12:
22	22	2008	5	5	Turkish Grand Prix	2024-07-21T09:58:36.897+00:...	2008-05-11T12:
23	23	2008	6	6	Monaco Grand Prix	2024-07-21T09:58:36.897+00:...	2008-05-25T12:
24	24	2008	7	7	Canadian Grand Prix	2024-07-21T09:58:36.897+00:...	2008-06-08T17:
25	25	2008	8	8	French Grand Prix	2024-07-21T09:58:36.897+00:...	2008-06-22T12:
26	26	2008	9	9	British Grand Prix	2024-07-21T09:58:36.897+00:...	2008-07-06T12:
27	27	2008	10	10	German Grand Prix	2024-07-21T09:58:36.897+00:...	2008-07-20T12:
28	28	2008	11	11	Hungarian Grand Prix	2024-07-21T09:58:36.897+00:...	2008-08-03T12:
29	29	2008	12	12	European Grand Prix	2024-07-21T09:58:36.897+00:...	2008-08-24T12:
30	30	2008	13	13	Belgian Grand Prix	2024-07-21T09:58:36.897+00:...	2008-09-07T12:
31	31	2008	14	14	Italian Grand Prix	2024-07-21T09:58:36.897+00:...	2008-09-14T12:
32	32	2008	15	15	Singapore Grand Prix	2024-07-21T09:58:36.897+00:...	2008-09-28T12:
33	33	2008	16	16	Japanese Grand Prix	2024-07-21T09:58:36.897+00:...	2008-10-12T04:
34	34	2008	17	17	Chinese Grand Prix	2024-07-21T09:58:36.897+00:...	2008-10-19T07:
35	35	2008	18	18	Brazilian Grand Prix	2024-07-21T09:58:36.897+00:...	2008-11-02T17:
36	36	2007	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:	2007-03-18T03:

1,058 rows

```
races_filtered_df = races_df.filter("race_year = 2019 and round <= 5")
```

```
display(races_filtered_df)
```

Table

	race_id	race_year	round	circuit_id	name	ingestion_date	race_timestam
1	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:
2	1011	2019	2	3	Bahrain Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-31T15:10:
3	1012	2019	3	17	Chinese Grand Prix	2024-07-21T09:58:36.897+00:...	2019-04-14T06:10:
4	1013	2019	4	73	Azerbaijan Grand P...	2024-07-21T09:58:36.897+00:...	2019-04-28T12:10:
5	1014	2019	5	4	Spanish Grand Pri	2024-07-21T09:58:36.897+00:	2019-05-12T12:10:

5 rows

```
races_filtered_df = races_df.filter((races_df["race_year"] == 2019) & (races_df["round"] <= 5))
```

```
display(races_filtered_df)
```

Table

	race_id	race_year	round	circuit_id	name	ingestion_date	race_timestam
1	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:
2	1011	2019	2	3	Bahrain Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-31T15:10:
3	1012	2019	3	17	Chinese Grand Prix	2024-07-21T09:58:36.897+00:...	2019-04-14T06:10:
4	1013	2019	4	73	Azerbaijan Grand P...	2024-07-21T09:58:36.897+00:...	2019-04-28T12:10:
5	1014	2019	5	4	Spanish Grand Pri...	2024-07-21T09:58:36.897+00:...	2019-05-12T12:10:

5 rows

(https://databricks.com)

```
select * from f1_presentation.calculated_race_results;
```

Table

	race_year	team_name	driver_name	position	points	calculated_points
1	1952	Ferrari	Alberto Ascari	1	9	10
2	1952	Ferrari	Nino Farina	2	6	9
3	1952	Ferrari	Piero Taruffi	3	4	8
4	1952	Gordini	Robert Manzon	4	3	7
5	1952	Simca	Maurice Trintignant	5	2	6
6	1952	HWM	Peter Collins	6	0	5
7	1952	Gordini	Jean Behra	7	0	4
8	1952	Maserati	Philippe Étancelin	8	0	3
9	1952	HWM	Lance Macklin	9	0	2
10	1952	HWM	Yves Cabantous	10	0	1
11	1996	Williams	Damon Hill	1	10	10
12	1996	Williams	Jacques Villeneuve	2	6	9
13	1996	Benetton	Jean Alesi	3	4	8
14	1996	McLaren	David Coulthard	4	3	7
15	1996	McLaren	Mika Häkkinen	5	2	6

10,000+ rows | Truncated data due to row limit

```
SELECT driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
GROUP BY driver_name
ORDER BY avg_points DESC
```

Table

	driver_name	total_races	total_points	avg_points
1	Bill Vukovich	2	20	10
2	Bill Holland	1	9	9
3	Luigi Fagioli	6	54	9
4	George Amick	1	9	9
5	Dorino Serafini	1	9	9
6	Juan Fangio	43	385	8.953488372093023
7	Alberto Ascari	22	194	8.818181818181818
8	Sam Hanks	4	35	8.75
9	Alain Prost	138	1156	8.376811594202898
10	Ayrton Senna	104	869	8.35576923076923
11	Jackie Stewart	61	507	8.311475409836065
12	Lewis Hamilton	232	1903	8.202586206896552
13	Nino Farina	27	221	8.185185185185185
14	Jim Clark	46	373	8.108695652173912
15	José Froilán González	19	152	8

506 rows

```
SELECT driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
GROUP BY driver_name
HAVING COUNT(1) >= 50
ORDER BY avg_points DESC
```

Table

	driver_name	total_races	total_points	avg_points
1	Alain Prost	138	1156	8.376811594202898
2	Ayrton Senna	104	869	8.35576923076923
3	Jackie Stewart	61	507	8.311475409836065
4	Lewis Hamilton	232	1903	8.202586206896552
5	Michael Schumacher	226	1796	7.946902654867257
6	Nigel Mansell	90	713	7.92222222222222...
7	Niki Lauda	81	639	7.888888888888889
8	Sebastian Vettel	203	1565	7.70935960591133
9	Damon Hill	70	520	7.428571428571429
10	Juan Pablo Montoya	58	427	7.362068965517241
11	Nelson Piquet	115	845	7.34782608695652...
12	Mika Häkkinen	96	694	7.229166666666667
13	Jack Brabham	67	473	7.059701492537314
14	Bruce McLaren	58	404	6.96551724137931...
15	Denny Hulme	72	497	6.902777777777778

64 rows

```
SELECT driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
 WHERE race_year BETWEEN 2011 AND 2020
GROUP BY driver_name
HAVING COUNT(1) >= 50
ORDER BY avg_points DESC
```

Table

	driver_name	total_races	total_points	avg_points
1	Lewis Hamilton	177	1478	8.350282485875706
2	Sebastian Vettel	164	1282	7.817073170731708
3	Nico Rosberg	93	673	7.236559139784946
4	Max Verstappen	88	605	6.875
5	Valtteri Bottas	117	793	6.777777777777778
6	Fernando Alonso	95	584	6.147368421052631
7	Kimi Räikkönen	121	721	5.958677685950414
8	Daniel Ricciardo	111	639	5.756756756756757
9	Jenson Button	69	377	5.463768115942029
10	Felipe Massa	94	427	4.542553191489362
11	Romain Grosjean	59	244	4.135593220338983
12	Sergio Pérez	116	446	3.84482758620689...
13	Carlos Sainz	65	244	3.753846153846154
14	Nico Hülkenberg	90	329	3.655555555555555

(https://databricks.com)

Ingest circuits.csv file

```
dbutils.widgets.help()
```

dbutils.widgets provides utilities for working with notebook widgets. You can create different types of widgets and get their bound value. For more info about a method, use **dbutils.widgets.help("methodName")**.

combobox(name: String, defaultValue: String, choices: Seq, label: String): void -> Creates a combobox input widget with a given name, default value and choices
dropdown(name: String, defaultValue: String, choices: Seq, label: String): void -> Creates a dropdown input widget a with given name, default value and choices
get(name: String): String -> Retrieves current value of an input widget
getAll: Map -> Retrieves a mapping of all current values of the input widgets
getArgument(name: String, optional: String): String -> (DEPRECATED) Equivalent to get
multiselect(name: String, defaultValue: String, choices: Seq, label: String): void -> Creates a multiselect input widget with a given name, default value and choices
remove(name: String): void -> Removes an input widget from the notebook
removeAll: void -> Removes all widgets in the notebook
text(name: String, defaultValue: String, label: String): void -> Creates a text input widget with a given name and default value

```
dbutils.widgets.text("p_data_sourec", "")
v_data_sourec = dbutils.widgets.get("p_data_sourec")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

either above or below command with full path will work

```
# %run /Workspace/Formula1/includes/configuration
```

```
# Databricks notebook source
raw_folder_path = "/mnt/formula1dl1204/raw"
processed_folder_path = "/mnt/formula1dl1204/processed"
presentation_folder_path = "/mnt/formula1dl1204/presentation"

#raw_folder_path = 'abfss://raw@formula1dl.dfs.core.windows.net'
#processed_folder_path = 'abfss://processed@formula1dl.dfs.core.windows.net'
#presentation_folder_path = 'abfss://presentation@formula1dl.dfs.core.windows.net'
```

Step1 - read the csv file using spark dataframe reader

```
display(dbutils.fs.mounts())
```

Table

	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl1204/presentati...	abfss://presentation@formula1dl1204.dfs.core.windows.n...	
3	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.net/	
4	/mnt/formula1dl1204/processed	abfss://processed@formula1dl1204.dfs.core.windows.net/	
5	/Volumes	UnityCatalogVolumes	
6	/mnt/formula1dl1204/raw	abfss://raw@formula1dl1204.dfs.core.windows.net/	
7	/databricks/mlflow-tracking	databricks/mlflow-tracking	
8	/databricks-results	databricks-results	
9	/databricks/mlflow-registry	databricks/mlflow-registry	
10	/Volume	DbfsReserved	
11	/mnt/formula1dl1204/demo	abfss://demo@formula1dl1204.dfs.core.windows.net/	
12	/volumes	DbfsReserved	
13	/	DatabricksRoot	
14	/volume	DbfsReserved	

14 rows

```
%fs
ls /mnt/formula1dl1204/raw
```

Table

	path	name	size	modificationTime
1	dbfs:/mnt/formula1dl1204/raw/2021-03-2...	2021-03-21/	0	1721668310000
2	dbfs:/mnt/formula1dl1204/raw/2021-03-2...	2021-03-28/	0	1721668310000
3	dbfs:/mnt/formula1dl1204/raw/2021-04-1...	2021-04-18/	0	1721668310000

3 rows

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType
```

```
circuits_schema = StructType(fields = [StructField("circuitId", IntegerType(), False),
                                       StructField("circuitRef", StringType(), True),
                                       StructField("name", StringType(), True),
                                       StructField("location", StringType(), True),
                                       StructField("country", StringType(), True),
                                       StructField("lat", DoubleType(), True),
                                       StructField("lng", DoubleType(), True),
                                       StructField("alt", IntegerType(), True),
                                       StructField("url", StringType(), True)
 ])
```

```
# circuits_df = spark.read \
#     .option("header", True) \
#     .option("inferSchema", True) \
#     .csv(f"{raw_folder_path}/{v_file_date}/circuits.csv")
```

```
circuits_df = spark.read \
    .option("header", True) \
    .schema(circuits_schema) \
    .csv(f"{raw_folder_path}/{v_file_date}/circuits.csv")
```

```
# type(circuits_df)
```

```
# circuits_df.show()
```

```
display(circuits_df)
```

Table

	circuitId	circuitRef	name	location	country	lat	lng
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.96
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.73
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.510
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.2611
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.40
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.4205
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.522
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.1636
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.0169
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	8.5658
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.248
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.33166
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.9713
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.2811

77 rows

```
# circuits_df.printSchema()
```

```
# circuits_df.describe().show()
```

Step2 - Select only the required columns

```
circuits_selected_df = circuits_df.select("circuitId", "circuitRef", "name", "location", "country", "lat", "lng", "alt")
```

```
display(circuits_selected_df)
```

Table

	circuitId	circuitRef	name	location	country	lat	lng
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.96
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.73
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.510
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.2611
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.40
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.4205
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.522
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.1636
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.0169
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	8.5658
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.248
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.33166
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.9713
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.2811

77 rows

```
circuits_selected_df = circuits_df.select(circuits_df.circuitId, circuits_df.circuitRef, circuits_df.name, circuits_df.location, ci
```

```
display(circuits_selected_df)
```

Table

	circuitId	circuitRef	name	location	country	lat	lng
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.96
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.73
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.510
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.2611
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.40
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.4205
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.522
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.1636
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.0169
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	8.5658
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.248
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.33166
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.9713
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.2811

77 rows

```
circuits_selected_df = circuits_df.select(circuits_df["circuitId"], circuits_df["circuitRef"], circuits_df["name"],
                                          circuits_df["location"], circuits_df["country"], circuits_df["lat"], circuits_df["lng"], circuits_d
```

```
from pyspark.sql.functions import col
```

```
#circuits_selected_df = circuits_df.select(col("circuitId"), col("circuitRef"), col("name"), col("location"), col("country").alias(
```

```
display(circuits_selected_df)
```

Table

	circuitId	circuitRef	name	location	country	lat	lng
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.96
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.73
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.510
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.2611
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.40
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.4205
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.522
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.1636
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.0169
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	8.5658
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.248
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.33166
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.9713
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.2811

77 rows

Step3 - Rename the columns as required

```
from pyspark.sql.functions import lit
```

```
circuits_renamed_df = circuits_selected_df \
  .withColumnRenamed("circuitId", "circuit_id") \
  .withColumnRenamed("circuitRef", "circuit_ref") \
  .withColumnRenamed("lat", "latitude") \
  .withColumnRenamed("lng", "longitude") \
  .withColumnRenamed("alt", "altitude") \
  .withColumn("data_source", lit(v_data_sourec)) \
  .withColumn("file_date", lit(v_file_date))
```

```
display(circuits_renamed_df)
```

Table

	circuit_id	circuit_ref	name	location	country	latitude	lon
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	

77 rows

Step4 - Add ingestion date to dataframe

```
#from pyspark.sql.functions import current_timestamp
```

```
#circuits_final_df = circuits_renamed_df.withColumn("ingestion_date", current_timestamp())
```

```
circuits_final_df = add_ingestion_date(circuits_renamed_df)
```

```
display(circuits_final_df)
```

Table

	circuit_id	circuit_ref	name	location	country	latitude	lon
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	

77 rows

suppose we need to add another column environment

-- not part of project, just additional info

```
#from pyspark.sql.functions import lit
```

```
#circuits_final_df = circuits_renamed_df.withColumn("ingestion_date", current_timestamp()) \
#    .withColumn("environment", lit("production"))
```

```
#display(circuits_final_df)
```

Step5 - Write data to datalake as parquet

```
circuits_final_df.write.mode("overwrite").format("delta").saveAsTable("f1_processed.circuits")
```

```
%sql
SELECT * FROM f1_processed.circuits
```

Table

	circuit_Id	circuit_ref	name	location	country	latitude	lon
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	

77 rows

```
display(dbutils.fs.mounts())
```

Table

	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl1204/presentati...	abfss://presentation@formula1dl1204.dfs.core.windows.n...	

3	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.net/	
4	/mnt/formula1dl1204/processed	abfss://processed@formula1dl1204.dfs.core.windows.net/	
5	/Volumes	UnityCatalogVolumes	
6	/mnt/formula1dl1204/raw	abfss://raw@formula1dl1204.dfs.core.windows.net/	
7	/databricks/mlflow-tracking	databricks/mlflow-tracking	
8	/databricks-results	databricks-results	
9	/databricks/mlflow-registry	databricks/mlflow-registry	
10	/Volume	DbfsReserved	
11	/mnt/formula1dl1204/demo	abfss://demo@formula1dl1204.dfs.core.windows.net/	
12	/volumes	DbfsReserved	
13	/	DatabricksRoot	
14	/volume	DbfsReserved	

14 rows

```
%sql
DESC  DATABASE EXTENDED f1_processed;
```

Table

	database_description_item	database_description_value
1	Catalog Name	spark_catalog
2	Namespace Name	f1_processed
3	Comment	
4	Location	dbfs:/mnt/formula1dl1204/processed
5	Owner	root
6	Properties	

6 rows

```
#display(spark.read.parquet(f"{processed_folder_path}/circuits"))
```

```
#df = spark.read.parquet(f"{processed_folder_path}/circuits")
```

```
%sql
select * from f1_processed.circuits;
```

Table

	circuit_Id	circuit_ref	name	location	country	1.2 latitude	1.2 lon
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	
7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	

Drop all the tables

```
DROP DATABASE IF EXISTS f1_processed CASCADE;
```

```
OK
```

```
CREATE DATABASE IF NOT EXISTS f1_processed
LOCATION "/mnt/formula1dl1204/processed";
```

```
OK
```

```
DROP DATABASE IF EXISTS f1_presentation CASCADE;
```

```
OK
```

```
CREATE DATABASE IF NOT EXISTS f1_presentation
LOCATION "/mnt/formula1dl1204/presentation";
```

```
OK
```

databricks trans / 1.race_results

(https://databricks.com)

Read all data from all data sources as required

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
# drivers_df = spark.read.parquet(f"{processed_folder_path}/drivers") \
#     .withColumnRenamed("name", "driver_name") \
#     .withColumnRenamed("number", "driver_number") \
#     .withColumnRenamed("nationality", "driver_nationality")
```

```
drivers_df = spark.read.format("delta").load(f"{processed_folder_path}/drivers") \
    .withColumnRenamed("number", "driver_number") \
    .withColumnRenamed("name", "driver_name") \
    .withColumnRenamed("nationality", "driver_nationality")
```

```
# constructors_df = spark.read.parquet(f"{processed_folder_path}/constructors") \
#     .withColumnRenamed("name", "team")
```

```
constructors_df = spark.read.format("delta").load(f"{processed_folder_path}/constructors") \
.withColumnRenamed("name", "team")
```

```
circuits_df = spark.read.format("delta").load(f"{processed_folder_path}/circuits") \
.withColumnRenamed("location", "circuit_location")
```

```
# circuits_df = spark.read.parquet(f"{processed_folder_path}/circuits") \
#     .withColumnRenamed("location", "circuit_location") \
#     .withColumnRenamed("circuit_Id", "circuit_id")
```

```
# races_df = spark.read.parquet(f"{processed_folder_path}/races") \
#     .withColumnRenamed("name", "race_name") \
#     .withColumnRenamed("race_timestamp", "race_date")
```

```
races_df = spark.read.format("delta").load(f"{processed_folder_path}/races") \
.withColumnRenamed("name", "race_name") \
.withColumnRenamed("race_timestamp", "race_date")
```

```
# results_df = spark.read.parquet(f"{processed_folder_path}/results") \
#     .withColumnRenamed("time", "race_time")
```

```
results_df = spark.read.format("delta").load(f"{processed_folder_path}/results") \
.filter(f"file_date = '{v_file_date}'") \
.withColumnRenamed("time", "race_time") \
.withColumnRenamed("race_id", "result_race_id") \
.withColumnRenamed("file_date", "result_file_date")
```

Join circuits to races

```
race_circuits_df = races_df.join(circuits_df, races_df.circuit_id == circuits_df.circuit_Id, "inner") \
    .select(races_df.race_id, races_df.race_year, races_df.race_name, races_df.race_date, circuits_df.circuit_location)
```

```
# circuits_df.show()
```

```
------------------+
|         1|   albert_park|Albert Park Grand...|       Melbourne|Australia|-37.8497|  144.968|      10|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         2|        sepang|Sepang Internatio...|    Kuala Lumpur| Malaysia| 2.76083|  101.738|      18|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         3|       bahrain|Bahrain Internati...|          Sakhir|  Bahrain| 26.0325|  50.5106|       7|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         4|     catalunya|Circuit de Barcel...|        Montmeló|    Spain|   41.57|  2.26111|     109|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         5|      istanbul|       Istanbul Park|        Istanbul|   Turkey| 40.9517|   29.405|     130|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         6|        monaco|   Circuit de Monaco|     Monte-Carlo|   Monaco| 43.7347|  7.42056|       7|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         7|    villeneuve|Circuit Gilles Vi...|        Montreal|   Canada|    45.5| -73.5228|      13|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         8|   magny_cours|Circuit de Nevers...|     Magny Cours|   France| 46.8642|  3.16361|     228|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|         9|   silverstone| Silverstone Circuit|     Silverstone|       UK| 52.0786| -1.01694|     153|     Ergast|2021-03-21|20
24-07-25 06:54:...|
|        10|hockenheimring|      Hockenheimring|      Hockenheim|  Germany| 49.3278|  8.56583|     103|     Ergast|2021-03-21|20
24-07-25 06:54:...|
```

```
# display(race_circuits_df)
```

Table

	race_id	race_year	race_name	race_date	circuit_location
21	1071	2021	Brazilian Grand Prix	2021-11-07T17:00:00.000+00:...	São Paulo
22	1072	2021	Saudi Arabian Grand Prix	2021-12-05T16:00:00.000+00:...	Jeddah

22	1072	2021	Saudi Arabian Grand Prix	2021-12-05T16:00:00.000+00:...	Jeddah
23	1073	2021	Abu Dhabi Grand Prix	2021-12-12T13:00:00.000+00:...	Abu Dhabi
24	989	2018	Australian Grand Prix	2018-03-25T05:10:00.000+00:...	Melbourne
25	990	2018	Bahrain Grand Prix	2018-04-08T15:10:00.000+00:...	Sakhir
26	991	2018	Chinese Grand Prix	2018-04-15T06:10:00.000+00:...	Shanghai
27	992	2018	Azerbaijan Grand Prix	2018-04-29T12:10:00.000+00:...	Baku
28	993	2018	Spanish Grand Prix	2018-05-13T13:10:00.000+00:...	Montmeló
29	994	2018	Monaco Grand Prix	2018-05-27T13:10:00.000+00:...	Monte-Carlo
30	995	2018	Canadian Grand Prix	2018-06-10T18:10:00.000+00:...	Montreal
31	996	2018	French Grand Prix	2018-06-24T14:10:00.000+00:...	Le Castellet
32	997	2018	Austrian Grand Prix	2018-07-01T13:10:00.000+00:...	Spielburg
33	998	2018	British Grand Prix	2018-07-08T13:10:00.000+00:...	Silverstone
34	999	2018	German Grand Prix	2018-07-22T13:10:00.000+00:...	Hockenheim
35	1000	2018	Hungarian Grand Prix	2018-07-29T13:10:00.000+00:...	Budapest
36	1001	2018	Belgian Grand Prix	2018-08-26T13:10:00.000+00:...	Spa

1,058 rows

Now join the above dataframe with other dataframes

```
race_results_df = results_df.join(race_circuits_df, results_df.result_race_id == race_circuits_df.race_id) \
                    .join(drivers_df, drivers_df.driver_id == results_df.driver_id) \
                    .join(constructors_df, constructors_df.constructor_id == results_df.constructor_id)
```

```
from pyspark.sql.functions import current_timestamp
```

```
final_df = race_results_df.select("race_id", "race_year", "race_name", "race_date", "circuit_location", "driver_name", "driver_numb
                                  "team", "grid", "fastest_lap", "race_time", "points", "position", "result_file_date") \
                          .withColumn("created_date", current_timestamp()) \
                          .withColumnRenamed("result_file_date", "file_date")
```

```
# display(final_df.filter("race_year == 2020 and race_name == 'Abu Dhabi Grand Prix'").orderBy(final_df.points.desc()))
```

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_presentation.race_results")
```

```
merge_condition = "tgt.driver_name = src.driver_name AND tgt.race_id = src.race_id"
merge_delta_data(final_df, 'f1_presentation', 'race_results', presentation_folder_path, merge_condition, 'race_id')
```

```
# %sql
# select * from f1_presentation.race_results;
```

Table

	race_id	race_year	race_name	race_date	circuit_location	driver_name	dr
1	873	2012	Singapore Grand P...	2012-09-23T12:00:00.000+00:...	Marina Bay	Charles Pic	

```
client_id = "24cd3bb8-d074-4488-b9d6-e4fc1e987054"
tenant_id = "d985f1c9-e414-4960-ad33-a2c5f015be99"
client_secret = "lL68Q~KFTYcurCdY2F6CgYVEeP7BSnpqObDk3bey"
```

```
spark.conf.set("fs.azure.account.auth.type.formula1dl1204.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.formula1dl1204.dfs.core.windows.net", "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.formula1dl1204.dfs.core.windows.net", client_id)
spark.conf.set("fs.azure.account.oauth2.client.secret.formula1dl1204.dfs.core.windows.net", client_secret)
spark.conf.set("fs.azure.account.oauth2.client.endpoint.formula1dl1204.dfs.core.windows.net", f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")
```

```
display(dbutils.fs.ls("abfss://demo@formula1dl1204.dfs.core.windows.net"))
```

Table

	path	name	size	modificationTime
1	abfss://demo@formula1dl1204.dfs.core.windows.net/circuits.c...	circuits.csv	10044	1721470200000

1 row

```
display(spark.read.csv("abfss://demo@formula1dl1204.dfs.core.windows.net/circuits.csv"))
```

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8
1	circuitId	circuitRef	name	location	country	lat	lng	alt	url
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10	http://en.wikipedia.org/wiki/Melbourne_Grand_Prix_Circuit
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18	http://en.wikipedia.org/wiki/Sepang_International_Circuit
4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7	http://en.wikipedia.org/wiki/Bahrain_International_Circuit
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109	http://en.wikipedia.org/wiki/Circuit_de_Barcelona-Catalunya

databricks trans / 2.driver_standings

(https://databricks.com)

```
dbutils.widgets.text("p_file_date", "2021-03-28")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/common_functions"
```

```
%run "../includes/configuration"
```

```
# race_results_df = spark.read.parquet(f"{presentation_folder_path}/race_results")
```

```
race_results_df = spark.read.format("delta").load(f"{presentation_folder_path}/race_results") \
.filter(f"file_date = '{v_file_date}'")
```

```
# display(race_results_df)
```

Table

	race_id	race_year	race_name	race_date	circuit_location	driver_name
1	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Mick Schumacher
2	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Nikita Mazepin
3	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Yuki Tsunoda
4	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Nicholas Latifi
5	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	George Russell
6	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Lando Norris
7	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Charles Leclerc
8	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Pierre Gasly
9	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Antonio Giovinazzi
10	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Lance Stroll
11	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Esteban Ocon
12	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Carlos Sainz
13	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Max Verstappen
14	1053	2021	Emilia Romagna Grand P...	2021-04-18T13:00:00.000+00:...	Imola	Valtteri Bottas

20 rows

```
race_year_list = df_column_to_list(race_results_df, 'race_year')
```

```
from pyspark.sql.functions import col

race_results_df = spark.read.format("delta").load(f"{presentation_folder_path}/race_results") \
.filter(col("race_year").isin(race_year_list))
```

```
from pyspark.sql.functions import sum, when, count, col

# driver_standings_df = race_results_df \
# .groupBy("race_year", "driver_name", "driver_nationality", "team") \
# .agg(sum("points").alias("total_points"),
#      count(when(col("position") == 1, True)).alias("wins"))

driver_standings_df = race_results_df \
.groupBy("race_year", "driver_name", "driver_nationality") \
.agg(sum("points").alias("total_points"),
     count(when(col("position") == 1, True)).alias("wins"))
```

```
from pyspark.sql.window import Window
from pyspark.sql.functions import desc, rank, asc
```

```
driver_rank_spec = Window.partitionBy("race_year").orderBy(desc("total_points"), desc("wins"))
final_df = driver_standings_df.withColumn("rank", rank().over(driver_rank_spec))
```

```
# from pyspark.sql.functions import sum

# driver_standings_df = race_results_df \
#     .groupBy("race_year", "driver_name", "driver_nationality", "team") \
#     .agg(sum("points").alias("total_points"))
```

```
# display(driver_standings_df.filter("race_year = 2020"))
```

```
# display(driver_standings_df.filter("race_year = 2020"))
```

```
# from pyspark.sql.window import Window
# from pyspark.sql.functions import desc, rank, asc

# driver_rank_spec = Window.partitionBy("race_year").orderBy(desc("total_points"))
# final_df = driver_standings_df.withColumn("rank", rank().over(driver_rank_spec))
```

```
# display(final_df.filter("race_year = 2020"))
```

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_presentation.driver_standings")
```

```
merge_condition = "tgt.driver_name = src.driver_name AND tgt.race_year = src.race_year"
merge_delta_data(final_df, 'f1_presentation', 'driver_standings', presentation_folder_path, merge_condition, 'race_year')
```

```
# %sql
# select * from f1_presentation.driver_standings;
```

Table

	race_year	driver_name	driver_nationality	total_points	wins	rank
1	1952	Alberto Ascari	Italian	53.5	6	1
2	1952	Nino Farina	Italian	27	0	2
3	1952	Piero Taruffi	Italian	22	1	3
4	1952	Mike Hawthorn	British	10	0	4
5	1952	Rudi Fischer	Swiss	10	0	4
6	1952	Robert Manzon	French	9	0	6
7	1952	Troy Ruttman	American	8	1	7
8	1952	Luigi Villoresi	Italian	8	0	8
9	1952	José Froilán González	Argentine	6.5	0	9
10	1952	Jean Behra	French	6	0	10
11	1952	Jim Rathmann	American	6	0	10
12	1952	Sam Hanks	American	4	0	12
13	1952	Dennis Poore	British	3	0	13
14	1952	Ken Wharton	British	3	0	13
15	1952	Duane Carter	American	3	0	13

3,142 rows

```
# %sql
# SELECT * FROM f1_presentation.driver_standings WHERE race_year = 2021;
```

Table

	race_year	driver_name	driver_nationality	total_points	wins	rank
1	2021	Lewis Hamilton	British	44	1	1
2	2021	Max Verstappen	Dutch	43	1	2
3	2021	Lando Norris	British	27	0	3
4	2021	Charles Leclerc	Monegasque	20	0	4
5	2021	Valtteri Bottas	Finnish	16	0	5
6	2021	Daniel Ricciardo	Australian	14	0	6
7	2021	Carlos Sainz	Spanish	14	0	6
8	2021	Sergio Pérez	Mexican	10	0	8
9	2021	Pierre Gasly	French	6	0	9
10	2021	Lance Stroll	Canadian	5	0	10
11	2021	Esteban Ocon	French	2	0	11
12	2021	Yuki Tsunoda	Japanese	2	0	11
13	2021	Fernando Alonso	Spanish	1	0	13
14	2021	George Russell	British	0	0	14
15	2021	Antonio Giovinazzi	Italian	0	0	14

20 rows

```
SELECT team_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
GROUP BY team_name
HAVING COUNT(1) >= 100
ORDER BY avg_points DESC
```

Table

	team_name	total_races	total_points	avg_points
1	Mercedes	385	2931	7.612987012987013
2	Ferrari	1478	10500	7.104194857916103
3	McLaren	1032	6865	6.65213178294573...
4	Red Bull	433	2851	6.584295612009238
5	Williams	745	4759	6.387919463087249
6	Benetton	296	1806	6.101351351351352
7	Team Lotus	352	2100	5.965909090909091
8	Cooper-Climax	130	772	5.938461538461539
9	Brabham	235	1373	5.842553191489362
10	BRM	215	1212	5.637209302325582
11	Renault	385	2105	5.467532467532467
12	Maserati	179	950	5.307262569832402
13	Tyrrell	339	1749	5.15929203539823
14	Jordan	184	882	4.793478260869565
15	Ligier	248	1134	4.57258064516129

23 rows

```
SELECT team_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
 WHERE race_year BETWEEN 2011 AND 2020
GROUP BY team_name
HAVING COUNT(1) >= 100
ORDER BY avg_points DESC
```

Table

	team_name	total_races	total_points	avg_points
1	Mercedes	331	2580	7.794561933534744
2	Red Bull	313	2201	7.031948881789138
3	Ferrari	314	2095	6.671974522292993
4	McLaren	204	984	4.823529411764706
5	Williams	125	531	4.248
6	Force India	189	671	3.55026455026455...

(https://databricks.com)

Ingest races file

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType
```

Step1 - Creating schema and reading data intop the schema

```
races_schema = StructType(fields=[StructField("raceId", IntegerType(), False),
                                  StructField("year", IntegerType(), True),
                                  StructField("round", IntegerType(), True),
                                  StructField("circuitId", IntegerType(), True),
                                  StructField("name", StringType(), True),
                                  StructField("date", DateType(), True),
                                  StructField("time", StringType(), True),
                                  StructField("url", StringType(), True)
                                  ])
```

```
races_df = spark.read \
    .option("header", True) \
    .schema(races_schema) \
    .csv(f"{raw_folder_path}/{v_file_date}/races.csv")
```

```
display(races_df)
```

Table

	raceId	year	round	circuitId	name	date	time	url
1	1	2009	1	1	Australian Grand Prix	2009-03-29	06:00:00	http://en.wikipedia.org
2	2	2009	2	2	Malaysian Grand Prix	2009-04-05	09:00:00	http://en.wikipedia.org
3	3	2009	3	17	Chinese Grand Prix	2009-04-19	07:00:00	http://en.wikipedia.org

4	4	2009	4	3	Bahrain Grand Prix	2009-04-26	12:00:00	http://en.wikipedia.org
5	5	2009	5	4	Spanish Grand Prix	2009-05-10	12:00:00	http://en.wikipedia.org
6	6	2009	6	6	Monaco Grand Prix	2009-05-24	12:00:00	http://en.wikipedia.org
7	7	2009	7	5	Turkish Grand Prix	2009-06-07	12:00:00	http://en.wikipedia.org
8	8	2009	8	9	British Grand Prix	2009-06-21	12:00:00	http://en.wikipedia.org
9	9	2009	9	20	German Grand Prix	2009-07-12	12:00:00	http://en.wikipedia.org
10	10	2009	10	11	Hungarian Grand Prix	2009-07-26	12:00:00	http://en.wikipedia.org
11	11	2009	11	12	European Grand Prix	2009-08-23	12:00:00	http://en.wikipedia.org
12	12	2009	12	13	Belgian Grand Prix	2009-08-30	12:00:00	http://en.wikipedia.org
13	13	2009	13	14	Italian Grand Prix	2009-09-13	12:00:00	http://en.wikipedia.org
14	14	2009	14	15	Singapore Grand Prix	2009-09-27	12:00:00	http://en.wikipedia.org

1,058 rows

Step 2 - Add ingestion_date and race_timestamp columns to the dataframe

```
from pyspark.sql.functions import current_timestamp, to_timestamp, lit, concat, col
```

```
races_with_timestamp_df = races_df.withColumn("ingestion_date", current_timestamp()) \
                          .withColumn("race_timestamp", to_timestamp(concat(col('date'), lit(' '), col('time')), 'yyyy-MM-dd HH:m
                          .withColumn("data_source", lit(v_data_source)) \
                          .withColumn("file_date", lit(v_file_date))
```

```
display(races_with_timestamp_df)
```

Table

	raceId	year	round	circuitId	name	date	time	url
1	1	2009	1	1	Australian Grand Prix	2009-03-29	06:00:00	http://en.wikipedia.org
2	2	2009	2	2	Malaysian Grand Prix	2009-04-05	09:00:00	http://en.wikipedia.org
3	3	2009	3	17	Chinese Grand Prix	2009-04-19	07:00:00	http://en.wikipedia.org
4	4	2009	4	3	Bahrain Grand Prix	2009-04-26	12:00:00	http://en.wikipedia.org
5	5	2009	5	4	Spanish Grand Prix	2009-05-10	12:00:00	http://en.wikipedia.org
6	6	2009	6	6	Monaco Grand Prix	2009-05-24	12:00:00	http://en.wikipedia.org
7	7	2009	7	5	Turkish Grand Prix	2009-06-07	12:00:00	http://en.wikipedia.org
8	8	2009	8	9	British Grand Prix	2009-06-21	12:00:00	http://en.wikipedia.org
9	9	2009	9	20	German Grand Prix	2009-07-12	12:00:00	http://en.wikipedia.org
10	10	2009	10	11	Hungarian Grand Prix	2009-07-26	12:00:00	http://en.wikipedia.org
11	11	2009	11	12	European Grand Prix	2009-08-23	12:00:00	http://en.wikipedia.org
12	12	2009	12	13	Belgian Grand Prix	2009-08-30	12:00:00	http://en.wikipedia.org
13	13	2009	13	14	Italian Grand Prix	2009-09-13	12:00:00	http://en.wikipedia.org
14	14	2009	14	15	Singapore Grand Prix	2009-09-27	12:00:00	http://en.wikipedia.org

1,058 rows

```
races_with_ingestion_date_df = add_ingestion_date(races_with_timestamp_df)
```

Step 3 - Select only the required columns and renaming columns as required

```
races_selected_df = races_with_timestamp_df.select(col('raceId').alias('race_id'), col('year').alias('race_year'),col('round'),
                                                   col('circuitId').alias('circuit_id'),col('name'),col('ingestion_date'),col('race
```

```
display(races_selected_df)
```

Table

	race_id	race_year	round	circuit_id	name	ingestion_date	race_timest
1	1	2009	1	1	Australian Grand Prix	2024-07-24T14:29:58.534+00:...	2009-03-29T06:
2	2	2009	2	2	Malaysian Grand Prix	2024-07-24T14:29:58.534+00:...	2009-04-05T09:
3	3	2009	3	17	Chinese Grand Prix	2024-07-24T14:29:58.534+00:...	2009-04-19T07:
4	4	2009	4	3	Bahrain Grand Prix	2024-07-24T14:29:58.534+00:...	2009-04-26T12:
5	5	2009	5	4	Spanish Grand Prix	2024-07-24T14:29:58.534+00:...	2009-05-10T12:
6	6	2009	6	6	Monaco Grand Prix	2024-07-24T14:29:58.534+00:...	2009-05-24T12:
7	7	2009	7	5	Turkish Grand Prix	2024-07-24T14:29:58.534+00:...	2009-06-07T12:
8	8	2009	8	9	British Grand Prix	2024-07-24T14:29:58.534+00:...	2009-06-21T12:
9	9	2009	9	20	German Grand Prix	2024-07-24T14:29:58.534+00:...	2009-07-12T12:
10	10	2009	10	11	Hungarian Grand Prix	2024-07-24T14:29:58.534+00:...	2009-07-26T12:
11	11	2009	11	12	European Grand Prix	2024-07-24T14:29:58.534+00:...	2009-08-23T12:
12	12	2009	12	13	Belgian Grand Prix	2024-07-24T14:29:58.534+00:...	2009-08-30T12:
13	13	2009	13	14	Italian Grand Prix	2024-07-24T14:29:58.534+00:...	2009-09-13T12:
14	14	2009	14	15	Singapore Grand Prix	2024-07-24T14:29:58.534+00:...	2009-09-27T12:

1,058 rows

Step 4 - Write output to processed container in parquet format

```
# races_selected_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.races")
```

```
races_selected_df.write.mode("overwrite").partitionBy('race_year').format("delta").saveAsTable("f1_processed.races")
```

```
%fs
ls /mnt/formula1dl1204/processed/races
```

Table

	path	name	size	modificationTime
1	dbfs:/mnt/formula1dl1204/processed/races/_delta_log/	_delta_log/	0	1721831399000
2	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
3	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
4	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
5	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
6	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
7	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
8	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000

9	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
10	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
11	dbfs:/mnt/formula1dl1204/processed/races/race_year=195...	race_year=195...	0	1721831400000
12	dbfs:/mnt/formula1dl1204/processed/races/race_year=196...	race_year=196...	0	1721831401000
13	dbfs:/mnt/formula1dl1204/processed/races/race_year=196...	race_year=196...	0	1721831401000
14	dbfs:/mnt/formula1dl1204/processed/races/race_year=196...	race_year=196...	0	1721831401000
15	dbfs:/mnt/formula1dl1204/processed/races/race_year=196...	race_year=196...	0	1721831401000

73 rows

```
# df = spark.read.parquet(f'{processed_folder_path}/races')
# df
```

```
%sql
select * from f1_processed.races;
```

Table

	race_id	race_year	round	circuit_id	name	ingestion_date	race_time
1	1053	2021	2	21	Emilia Romagna Grand P...	2024-07-24T14:29:59.841+00:...	2021-04-18T
2	1052	2021	1	3	Bahrain Grand Prix	2024-07-24T14:29:59.841+00:...	2021-03-28T
3	1051	2021	21	1	Australian Grand Prix	2024-07-24T14:29:59.841+00:...	2021-11-21T
4	1054	2021	3	20	TBC	2024-07-24T14:29:59.841+00:...	null
5	1055	2021	4	4	Spanish Grand Prix	2024-07-24T14:29:59.841+00:...	2021-05-09T
6	1056	2021	5	6	Monaco Grand Prix	2024-07-24T14:29:59.841+00:...	2021-05-23T
7	1057	2021	6	73	Azerbaijan Grand Prix	2024-07-24T14:29:59.841+00:...	2021-06-06T
8	1058	2021	7	7	Canadian Grand Prix	2024-07-24T14:29:59.841+00:...	2021-06-13T
9	1059	2021	8	34	French Grand Prix	2024-07-24T14:29:59.841+00:...	2021-06-27T
10	1060	2021	9	70	Austrian Grand Prix	2024-07-24T14:29:59.841+00:...	2021-07-04T
11	1061	2021	10	9	British Grand Prix	2024-07-24T14:29:59.841+00:...	2021-07-18T
12	1062	2021	11	11	Hungarian Grand Prix	2024-07-24T14:29:59.841+00:...	2021-08-01T
13	1063	2021	12	13	Belgian Grand Prix	2024-07-24T14:29:59.841+00:...	2021-08-29T
14	1064	2021	13	39	Dutch Grand Prix	2024-07-24T14:29:59.841+00:...	2021-09-05T

1,058 rows

```
dbutils.notebook.exit("Success")
```

```
Success
```

Optional Step - Partitioning data based on race_year

```
#races_selected_df.write.mode("overwrite").partitionBy('race_year').parquet('/mnt/formula1dl1204/processed/races')
```

databricks demo / 2.join_demo

(https://databricks.com)

Inner Join

```
%run "../includes/configuration"
```

```
races_df = spark.read.parquet(f"{processed_folder_path}/races").filter("race_year = 2019") \
    .withColumnRenamed("name", "race_name")
```

```
circuits_df = spark.read.parquet(f"{processed_folder_path}/circuits") \
    .filter("circuit_id < 70") \
    .withColumnRenamed("name", "circuit_name")
```

```
display(races_df)
```

Table

	race_id	race_year	round	circuit_id	race_name	ingestion_date	race_timesta
1	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:
2	1011	2019	2	3	Bahrain Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-31T15:
3	1012	2019	3	17	Chinese Grand Prix	2024-07-21T09:58:36.897+00:...	2019-04-14T06:
4	1013	2019	4	73	Azerbaijan Grand Prix	2024-07-21T09:58:36.897+00:...	2019-04-28T12:
5	1014	2019	5	4	Spanish Grand Prix	2024-07-21T09:58:36.897+00:...	2019-05-12T13:
6	1015	2019	6	6	Monaco Grand Prix	2024-07-21T09:58:36.897+00:...	2019-05-26T13:
7	1016	2019	7	7	Canadian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-06-09T18:
8	1017	2019	8	34	French Grand Prix	2024-07-21T09:58:36.897+00:...	2019-06-23T13:
9	1018	2019	9	70	Austrian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-06-30T13:
10	1019	2019	10	9	British Grand Prix	2024-07-21T09:58:36.897+00:...	2019-07-14T13:
11	1020	2019	11	10	German Grand Prix	2024-07-21T09:58:36.897+00:...	2019-07-28T13:
12	1021	2019	12	11	Hungarian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-08-04T13:
13	1022	2019	13	13	Belgian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-09-01T13:
14	1023	2019	14	14	Italian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-09-08T13:

21 rows

```
display(circuits_df)
```

Table

	circuit_Id	circuit_ref	circuit_name	location	country	1.2 latitude	1.2 lon
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	
2	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	
3	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	
4	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	
5	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	
6	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	

7	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5
8	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642
9	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786
10	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278
11	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789
12	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589
13	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372
14	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156

69 rows

```
race_circuits_df = circuits_df.join(races_df, circuits_df.circuit_Id == races_df.circuit_id, "inner") \
    .select(circuits_df.circuit_name, circuits_df.location, circuits_df.country, races_df.race_name, races_df.round)
```

```
display(race_circuits_df)
```

Table

	circuit_name	location	country	race_name	round
1	Albert Park Grand Prix Circuit	Melbourne	Australia	Australian Grand Prix	1
2	Bahrain International Circuit	Sakhir	Bahrain	Bahrain Grand Prix	2
3	Shanghai International Circuit	Shanghai	China	Chinese Grand Prix	3
4	Circuit de Barcelona-Catalunya	Montmeló	Spain	Spanish Grand Prix	5
5	Circuit de Monaco	Monte-Carlo	Monaco	Monaco Grand Prix	6
6	Circuit Gilles Villeneuve	Montreal	Canada	Canadian Grand Prix	7
7	Circuit Paul Ricard	Le Castellet	France	French Grand Prix	8
8	Silverstone Circuit	Silverstone	UK	British Grand Prix	10
9	Hockenheimring	Hockenheim	Germany	German Grand Prix	11
10	Hungaroring	Budapest	Hungary	Hungarian Grand Prix	12
11	Circuit de Spa-Francorchamps	Spa	Belgium	Belgian Grand Prix	13
12	Autodromo Nazionale di Monza	Monza	Italy	Italian Grand Prix	14
13	Marina Bay Street Circuit	Marina Bay	Singapore	Singapore Grand Prix	15
14	Suzuka Circuit	Suzuka	Japan	Japanese Grand Prix	17
15	Autódromo Hermanos Rodríguez	Mexico City	Mexico	Mexican Grand Prix	18

18 rows

Outer Joins

```
# left_outer_join
race_circuits_df = circuits_df.join(races_df, circuits_df.circuit_Id == races_df.circuit_id, "left") \
    .select(circuits_df.circuit_name, circuits_df.location, circuits_df.country, races_df.race_name, races_df.round)
```

```
display(race_circuits_df)
```

Table

	circuit_name	location	country	race_name	round
1	Albert Park Grand Prix Circuit	Melbourne	Australia	Australian Grand Prix	1
2	Sepang International Circuit	Kuala Lumpur	Malaysia	null	null

3	Bahrain International Circuit	Sakhir	Bahrain	Bahrain Grand Prix	2
4	Circuit de Barcelona-Catalunya	Montmeló	Spain	Spanish Grand Prix	5
5	Istanbul Park	Istanbul	Turkey	null	null
6	Circuit de Monaco	Monte-Carlo	Monaco	Monaco Grand Prix	6
7	Circuit Gilles Villeneuve	Montreal	Canada	Canadian Grand Prix	7
8	Circuit de Nevers Magny-Cours	Magny Cours	France	null	null
9	Silverstone Circuit	Silverstone	UK	British Grand Prix	10
10	Hockenheimring	Hockenheim	Germany	German Grand Prix	11
11	Hungaroring	Budapest	Hungary	Hungarian Grand Prix	12
12	Valencia Street Circuit	Valencia	Spain	null	null
13	Circuit de Spa-Francorchamps	Spa	Belgium	Belgian Grand Prix	13
14	Autodromo Nazionale di Monza	Monza	Italy	Italian Grand Prix	14
15	Marina Bay Street Circuit	Marina Bay	Singapore	Singapore Grand Prix	15

69 rows

```
# right_outer_join
race_circuits_df = circuits_df.join(races_df, circuits_df.circuit_Id == races_df.circuit_id, "right") \
    .select(circuits_df.circuit_name, circuits_df.location, circuits_df.country, races_df.race_name, races_df.round)
```

```
display(race_circuits_df)
```

Table

	circuit_name	location	country	race_name	round
1	Albert Park Grand Prix Circuit	Melbourne	Australia	Australian Grand Prix	1
2	Bahrain International Circuit	Sakhir	Bahrain	Bahrain Grand Prix	2
3	Shanghai International Circuit	Shanghai	China	Chinese Grand Prix	3
4	null	null	null	Azerbaijan Grand Prix	4
5	Circuit de Barcelona-Catalunya	Montmeló	Spain	Spanish Grand Prix	5
6	Circuit de Monaco	Monte-Carlo	Monaco	Monaco Grand Prix	6
7	Circuit Gilles Villeneuve	Montreal	Canada	Canadian Grand Prix	7
8	Circuit Paul Ricard	Le Castellet	France	French Grand Prix	8
9	null	null	null	Austrian Grand Prix	9
10	Silverstone Circuit	Silverstone	UK	British Grand Prix	10
11	Hockenheimring	Hockenheim	Germany	German Grand Prix	11
12	Hungaroring	Budapest	Hungary	Hungarian Grand Prix	12
13	Circuit de Spa-Francorchamps	Spa	Belgium	Belgian Grand Prix	13
14	Autodromo Nazionale di Monza	Monza	Italy	Italian Grand Prix	14
15	Marina Bay Street Circuit	Marina Bay	Singapore	Singapore Grand Prix	15

21 rows

```
# full_outer_join
race_circuits_df = circuits_df.join(races_df, circuits_df.circuit_Id == races_df.circuit_id, "full") \
    .select(circuits_df.circuit_name, circuits_df.location, circuits_df.country, races_df.race_name, races_df.round)
```

```
display(race_circuits_df)
```

Table

	circuit_name	location	country	race_name	round
1	Albert Park Grand Prix Circuit	Melbourne	Australia	Australian Grand Prix	1
2	Sepang International Circuit	Kuala Lumpur	Malaysia	null	null
3	Bahrain International Circuit	Sakhir	Bahrain	Bahrain Grand Prix	2
4	Circuit de Barcelona-Catalunya	Montmeló	Spain	Spanish Grand Prix	5
5	Istanbul Park	Istanbul	Turkey	null	null
6	Circuit de Monaco	Monte-Carlo	Monaco	Monaco Grand Prix	6
7	Circuit Gilles Villeneuve	Montreal	Canada	Canadian Grand Prix	7
8	Circuit de Nevers Magny-Cours	Magny Cours	France	null	null
9	Silverstone Circuit	Silverstone	UK	British Grand Prix	10
10	Hockenheimring	Hockenheim	Germany	German Grand Prix	11
11	Hungaroring	Budapest	Hungary	Hungarian Grand Prix	12
12	Valencia Street Circuit	Valencia	Spain	null	null
13	Circuit de Spa-Francorchamps	Spa	Belgium	Belgian Grand Prix	13
14	Autodromo Nazionale di Monza	Monza	Italy	Italian Grand Prix	14
15	Marina Bay Street Circuit	Marina Bay	Singapore	Singapore Grand Prix	15

72 rows

Semi Join

It works as Inner Join but you will get data from the Left Table only

```
# semi_join
race_circuits_df = circuits_df.join(races_df, circuits_df.circuit_Id == races_df.circuit_id, "semi")
```

```
display(race_circuits_df)
```

Table

	circuit_Id	circuit_ref	circuit_name	location	country	latitude	longitude
1	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.9
2	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.51
3	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.261
4	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.420
5	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.52
6	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.016
7	10	hockenheimring	Hockenheimring	Hockenheim	Germany	49.3278	8.565
8	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.24
9	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.971
10	14	monza	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.281
11	15	marina_bay	Marina Bay Street Circuit	Marina Bay	Singapore	1.2914	103.8
12	17	shanghai	Shanghai International Circuit	Shanghai	China	31.3389	121.
13	18	interlagos	Autódromo José Carlos Pace	São Paulo	Brazil	-23.7036	-46.69
14	22	suzuka	Suzuka Circuit	Suzuka	Japan	34.8431	136.5

18 rows

Anti Join

It will give everything from the left dataframe which is not found in the right dataframe

```
# anti_join
race_circuits_df = circuits_df.join(races_df, circuits_df.circuit_Id == races_df.circuit_id, "anti")
```

```
display(race_circuits_df)
```

Table

	circuit_Id	circuit_ref	circuit_name	location	country	latitude	lon
1	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	
2	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	
3	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	
4	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	
5	16	fuji	Fuji Speedway	Oyama	Japan	35.3717	
6	19	indianapolis	Indianapolis Motor Speedway	Indianapolis	USA	39.795	
7	20	nurburgring	Nürburgring	Nürburg	Germany	50.3356	
8	21	imola	Autodromo Enzo e Dino Ferrari	Imola	Italy	44.3439	
9	23	osterreichring	A1-Ring	Spielburg	Austria	47.2197	
10	25	galvez	Autódromo Juan y Oscar Gálvez	Buenos Aires	Argentina	-34.6943	
11	26	jerez	Circuito de Jerez	Jerez de la Frontera	Spain	36.7083	
12	27	estoril	Autódromo do Estoril	Estoril	Portugal	38.7506	
13	28	okayama	Okayama International Circuit	Okayama	Japan	34.915	
14	29	adelaide	Adelaide Street Circuit	Adelaide	Australia	-34.9272	

51 rows

Cross Join

```
# cross_join
race_circuits_df = races_df.crossJoin(circuits_df)
```

```
display(race_circuits_df)
```

Table

	race_id	race_year	round	circuit_id	race_name	ingestion_date	race_timestamp
1	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
2	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
3	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
4	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
5	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
6	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
7	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
8	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
9	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
10	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
11	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
12	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0
13	1010	2019	1	1	Australian Grand Prix	2024-07-21T09:58:36.897+00:...	2019-03-17T05:10:0

```
%run "../includes/configuration"
```

Agrregate functions demo

Build in Aggregate functions

```
race_results_df = spark.read.parquet(f"{presentation_folder_path}/race_results")
```

```
display(race_results_df)
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
69	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Heikki Kovalainen	Finnish
70	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Mark Webber	Australian
71	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Nelson Piquet Jr.	Brazilian
72	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Nico Rosberg	German
73	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Fernando Alonso	Spanish
74	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Timo Glock	German
75	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Kimi Räikkönen	Finnish
76	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Rubens Barrichello	Brazilian
77	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Lewis Hamilton	British
78	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Jarno Trulli	Italian
79	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Sebastian Vettel	German
80	2009	Bahrain Grand Prix	2009-04-26T12:00:00.000+00:...	Sakhir	Jenson Button	British
81	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Adrian Sutil	German
82	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Sébastien Bourdais	French
83	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Sébastien Buemi	Swiss
84	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Jarno Trulli	Italian

10,000+ rows | Truncated data due to row limit

```
demo_df = race_results_df.filter("race_year == 2020")
```

```
from pyspark.sql.functions import count, countDistinct, sum
```

```
demo_df.select(count("*")).show()
```

```
+--------+
|count(1)|
+--------+
|     340|
+--------+
```

```
demo_df.select(count("race_name")).show()
```

```
+----------------+
|count(race_name)|
+----------------+
|             340|
+----------------+
```

```
demo_df.select(countDistinct("race_name")).show()
```

```
+-------------------------+
|count(DISTINCT race_name)|
+-------------------------+
|                       17|
+-------------------------+
```

```
demo_df.select(sum("points")).show()
```

```
+-----------+
|sum(points)|
+-----------+
|     1734.0|
+-----------+
```

sum of points scored by driver Lewis Hamilton

```
demo_df.filter("driver_name = 'Lewis Hamilton'").select(sum("points")).show()
```

```
+-----------+
|sum(points)|
+-----------+
|      347.0|
+-----------+
```

we can use multiple functions

```
demo_df.filter("driver_name = 'Lewis Hamilton'").select(sum("points").alias("total_points"), countDistinct("race_name").alias("numb
```

```
+------------+---------------+
|total_points|number_of_races|
+------------+---------------+
|       347.0|             16|
+------------+---------------+
```

Group By

```
demo_df \
    .groupBy("driver_name") \
    .sum("points")  \
    .show()
```

```
|   Kevin Magnussen|        1.0|
|      Sergio Pérez|      125.0|
|      Carlos Sainz|      105.0|
|    Kimi Räikkönen|        4.0|
|   Romain Grosjean|        2.0|
|   Charles Leclerc|       98.0|
|   Alexander Albon|      105.0|
|      Lance Stroll|       75.0|
|      Pierre Gasly|       75.0|
|    Lewis Hamilton|      347.0|
|   Nico Hülkenberg|       10.0|
|  Daniel Ricciardo|      119.0|
|   Valtteri Bottas|      223.0|
|Antonio Giovinazzi|        4.0|
|      Lando Norris|       97.0|
|    Max Verstappen|      214.0|
|    George Russell|        3.0|
|  Sebastian Vettel|       33.0|
+------------------+-----------+
only showing top 20 rows
```

Aggregations under Group By

```
demo_df \
    .groupBy("driver_name") \
    .agg(sum("points").alias("total_points"), countDistinct("race_name").alias("number_of_races")) \
    .show()
```

```
|   Kevin Magnussen|         1.0|              17|
|      Sergio Pérez|       125.0|              15|
|      Carlos Sainz|       105.0|              17|
|    Kimi Räikkönen|         4.0|              17|
|   Romain Grosjean|         2.0|              15|
|   Charles Leclerc|        98.0|              17|
|   Alexander Albon|       105.0|              17|
|      Lance Stroll|        75.0|              16|
|      Pierre Gasly|        75.0|              17|
|    Lewis Hamilton|       347.0|              16|
|   Nico Hülkenberg|        10.0|               3|
|  Daniel Ricciardo|       119.0|              17|
|   Valtteri Bottas|       223.0|              17|
|Antonio Giovinazzi|         4.0|              17|
|      Lando Norris|        97.0|              17|
|    Max Verstappen|       214.0|              17|
|    George Russell|         3.0|              17|
|  Sebastian Vettel|        33.0|              17|
+------------------+------------+---------------+
only showing top 20 rows
```

Windows Functions

```
demo_df = race_results_df.filter("race_year in (2019,2020)")
```

```
display(demo_df)
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Carlos Sainz	Spanish
2	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Daniel Ricciardo	Australian
3	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Romain Grosjean	French
4	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Robert Kubica	Polish
5	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	George Russell	British
6	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Antonio Giovinazzi	Italian
7	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Alexander Albon	Thai
8	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Sergio Pérez	Mexican
9	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Lando Norris	British
10	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Pierre Gasly	French
11	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Daniil Kvyat	Russian
12	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Lance Stroll	Canadian
13	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Kimi Räikkönen	Finnish
14	2019	Australian Grand Prix	2019-03-17T05:10:00.000+00:...	Melbourne	Nico Hülkenberg	German

760 rows

```
demo_grouped_df = demo_df \
    .groupBy("race_year", "driver_name") \
    .agg(sum("points").alias("total_points"), countDistinct("race_name").alias("number_of_races")) \
    .show()
```

```
|     2020|   Kevin Magnussen|         1.0|              17|
|     2020|Antonio Giovinazzi|         4.0|              17|
|     2020|   Nico Hülkenberg|        10.0|               3|
|     2020|   Romain Grosjean|         2.0|              15|
|     2019|     Robert Kubica|         1.0|              21|
|     2020|   Charles Leclerc|        98.0|              17|
|     2019|      Lance Stroll|        21.0|              21|
|     2020|      Esteban Ocon|        62.0|              17|
|     2020| Pietro Fittipaldi|         0.0|               2|
|     2019|      Lando Norris|        49.0|              21|
|     2019|  Sebastian Vettel|       240.0|              21|
|     2019|    George Russell|         0.0|              21|
|     2019|      Carlos Sainz|        96.0|              21|
|     2020|  Sebastian Vettel|        33.0|              17|
|     2020|  Daniel Ricciardo|       119.0|              17|
|     2020|    Kimi Räikkönen|         4.0|              17|
|     2019|    Max Verstappen|       278.0|              21|
|     2019|   Valtteri Bottas|       326.0|              21|
+---------+------------------+------------+----------------+
only showing top 20 rows
```

```
demo_grouped_df = demo_df \
    .groupBy("race_year", "driver_name") \
    .agg(sum("points").alias("total_points"), countDistinct("race_name").alias("number_of_races"))
```

```
display(demo_grouped_df)
```

Table

	race_year	driver_name	total_points	number_of_races
1	2020	Daniil Kvyat	32	17
2	2019	Kevin Magnussen	20	21
3	2020	Kevin Magnussen	1	17
4	2020	Antonio Giovinazzi	4	17
5	2020	Nico Hülkenberg	10	3
6	2020	Romain Grosjean	2	15
7	2019	Robert Kubica	1	21
8	2020	Charles Leclerc	98	17
9	2019	Lance Stroll	21	21
10	2020	Esteban Ocon	62	17
11	2020	Pietro Fittipaldi	0	2
12	2019	Lando Norris	49	21
13	2019	Sebastian Vettel	240	21
14	2019	George Russell	0	21
15	2019	Carlos Sainz	96	21

43 rows

Rank

```
from pyspark.sql.window import Window
from pyspark.sql.functions import desc, rank, dense_rank

driverRankSpec = Window.partitionBy("race_year").orderBy(desc("total_points"))
demo_grouped_df.withColumn("rank", rank().over(driverRankSpec)).show(100)
```

```
|     2020|  Daniel Ricciardo|       119.0|             17|   5|
|     2020|      Carlos Sainz|       105.0|             17|   6|
|     2020|   Alexander Albon|       105.0|             17|   6|
|     2020|   Charles Leclerc|        98.0|             17|   8|
|     2020|      Lando Norris|        97.0|             17|   9|
|     2020|      Pierre Gasly|        75.0|             17|  10|
|     2020|      Lance Stroll|        75.0|             16|  10|
|     2020|      Esteban Ocon|        62.0|             17|  12|
|     2020|  Sebastian Vettel|        33.0|             17|  13|
|     2020|      Daniil Kvyat|        32.0|             17|  14|
|     2020|   Nico Hülkenberg|        10.0|              3|  15|
|     2020|Antonio Giovinazzi|         4.0|             17|  16|
|     2020|    Kimi Räikkönen|         4.0|             17|  16|
|     2020|    George Russell|         3.0|             17|  18|
|     2020|   Romain Grosjean|         2.0|             15|  19|
|     2020|   Kevin Magnussen|         1.0|             17|  20|
|     2020| Pietro Fittipaldi|         0.0|              2|  21|
|     2020|       Jack Aitken|         0.0|              1|  21|
|     2020|   Nicholas Latifi|         0.0|             17|  21|
+---------+------------------+------------+---------------+----+
```

Dense Rank

```
from pyspark.sql.window import Window
from pyspark.sql.functions import desc, rank, dense_rank

driverRankSpec = Window.partitionBy("race_year").orderBy(desc("total_points"))
demo_grouped_df.withColumn("rank", dense_rank().over(driverRankSpec)).show(100)
```

```
|     2020|  Daniel Ricciardo|       119.0|             17|   5|
|     2020|      Carlos Sainz|       105.0|             17|   6|
|     2020|   Alexander Albon|       105.0|             17|   6|
|     2020|   Charles Leclerc|        98.0|             17|   7|
|     2020|      Lando Norris|        97.0|             17|   8|
|     2020|      Pierre Gasly|        75.0|             17|   9|
|     2020|      Lance Stroll|        75.0|             16|   9|
|     2020|      Esteban Ocon|        62.0|             17|  10|
|     2020|  Sebastian Vettel|        33.0|             17|  11|
|     2020|      Daniil Kvyat|        32.0|             17|  12|
|     2020|   Nico Hülkenberg|        10.0|              3|  13|
|     2020|Antonio Giovinazzi|         4.0|             17|  14|
|     2020|    Kimi Räikkönen|         4.0|             17|  14|
|     2020|    George Russell|         3.0|             17|  15|
|     2020|   Romain Grosjean|         2.0|             15|  16|
|     2020|   Kevin Magnussen|         1.0|             17|  17|
|     2020| Pietro Fittipaldi|         0.0|              2|  18|
|     2020|       Jack Aitken|         0.0|              1|  18|
|     2020|   Nicholas Latifi|         0.0|             17|  18|
+---------+------------------+------------+---------------+----+
```

Produce constructor standings

```
dbutils.widgets.text("p_file_date", "2021-03-28")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
# race_results_df = spark.read.parquet(f"{presentation_folder_path}/race_results")
```

```
race_results_df = spark.read.format("delta").load(f"{presentation_folder_path}/race_results") \
.filter(f"file_date = '{v_file_date}'")
```

```
race_year_list = df_column_to_list(race_results_df, 'race_year')
```

```
from pyspark.sql.functions import col

race_results_df = spark.read.format("delta").load(f"{presentation_folder_path}/race_results") \
.filter(col("race_year").isin(race_year_list))
```

```
from pyspark.sql.functions import sum, when, count, col

constructor_standings_df = race_results_df \
.groupBy("race_year", "team") \
.agg(sum("points").alias("total_points"),
     count(when(col("position") == 1, True)).alias("wins"))
```

```
# display(constructor_standings_df.filter("race_year = 2020"))
```

```
from pyspark.sql.window import Window
from pyspark.sql.functions import desc, rank, asc

constructor_rank_spec = Window.partitionBy("race_year").orderBy(desc("total_points"), desc("wins"))
final_df = constructor_standings_df.withColumn("rank", rank().over(constructor_rank_spec))
```

```
# display(final_df.filter("race_year = 2020"))
```

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_presentation.constructor_standings")
```

```
merge_condition = "tgt.team = src.team AND tgt.race_year = src.race_year"
merge_delta_data(final_df, 'f1_presentation', 'constructor_standings', presentation_folder_path, merge_condition, 'race_year')
```

```
# %sql
# select * from f1_presentation.constructor_standings;
```

Table

	race_year	team	total_points	wins	rank
64	1959	Cooper-Borgward	0	0	8
65	1959	Cooper-OSCA	0	0	8
66	1959	Aston Martin	0	0	8
67	1959	Cooper-Maserati	0	0	8
68	1959	Vanwall	0	0	8
69	1959	JBW	0	0	8
70	1989	McLaren	141	10	1
71	1989	Williams	77	2	2
72	1989	Ferrari	59	3	3
73	1989	Benetton	39	1	4
74	1989	Tyrrell	16	0	5
75	1989	Team Lotus	15	0	6
76	1989	Arrows	13	0	7
77	1989	Brabham	8	0	8
78	1989	Dallara	8	0	8
79	1989	Onyx	6	0	10

1,081 rows

Table

	race_year	count(1)
1	2021	10
2	2020	10
3	2019	10
4	2018	10
5	2017	10
6	2016	11
7	2015	10
8	2014	11
9	2013	11
10	2012	12
11	2011	12
12	2010	12
13	2009	10
14	2008	11

databricks Set-up / 3.explore_dbutils_secrets_utility

(https://databricks.com)

```
dbutils.secrets.help()
```

Provides utilities for leveraging secrets within notebooks. Databricks documentation for more info.

get(scope: String, key: String): String -> Gets the string representation of a secret value with scope and key
getBytes(scope: String, key: String): byte[] -> Gets the bytes representation of a secret value with scope and key
list(scope: String): Seq -> Lists secret metadata for secrets within a scope
listScopes: Seq -> Lists secret scopes

```
dbutils.secrets.listScopes()
```

```
[SecretScope(name='formula1-scope')]
```

```
dbutils.secrets.list(scope = 'formula1-scope')
```

```
[SecretMetadata(key='formula1dl-account-key')]
```

```
'[REDACTED]'
```

Ingest constructors.jason files

Step 1 - Read the Jason file using Spark dataframe reader

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
constructors_schema = "constructorId INT, constructorRef STRING, name STRING, nationality STRING, url STRING"
```

```
constructor_df = spark.read \
.schema(constructors_schema) \
.json(f"{raw_folder_path}/{v_file_date}/constructors.json")
```

```
# display(constructor_df)
```

Step 2 - Drop unwanted columns from the dataframe

```
from pyspark.sql.functions import col
```

```
constructor_dropped_df = constructor_df.drop(col('url'))
```

```
# display(constructor_dropped_df)
```

Step 3 - Rename columns and add ingestion date

```
from pyspark.sql.functions import current_timestamp, lit
```

```
constructor_renamed_df = constructor_dropped_df.withColumnRenamed("constructorId", "constructor_id") \
                                               .withColumnRenamed("constructorRef", "constructor_ref") \
                                               .withColumn("data_source", lit(v_data_source)) \
                                               .withColumn("file_date", lit(v_file_date))
```

```
constructor_final_df = add_ingestion_date(constructor_renamed_df)
```

```
# display(constructor_final_df)
```

Step 4 Write output to parquet file

```
# constructor_final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.constructors")
```

```
constructor_final_df.write.mode("overwrite").format("delta").saveAsTable("f1_processed.constructors")
```

```
%fs
ls /mnt/formula1dl1204/processed/constructors
```

Table

	path	name
1	dbfs:/mnt/formula1dl1204/processed/constructors/_delta_log/	_delta_log/
2	dbfs:/mnt/formula1dl1204/processed/constructors/part-00000-bcdf1ec2-45a6-4646-a177-e1ee9a9f18c6-c000.snappy.parqu...	part-00000-bcdf1ec2-45a6-4646-a17

2 rows

```
# df = spark.read.parquet(f"{processed_folder_path}/constructors")
```

```
# display(df)
```

```
%sql
desc extended f1_processed.constructors
```

Table

	col_name	data_type
1	constructor_id	int
2	constructor_ref	string
3	name	string
4	nationality	string
5	data_source	string
6	file_date	string
7	ingestion_date	timestamp
8		
9	# Delta Statistics Columns	
10	Column Names	name, ingestion_date, constructor_ref, constructor_id, file_date, nationality, data_source
11	Column Selection Method	first-32
12		
13	# Detailed Table Information	
14	Catalog	spark_catalog

25 rows

```
%sql
select * from f1_processed.constructors;
```

Table

	constructor_id	constructor_ref	name	nationality	data_source	file_date	ingestion_d
1	1	mclaren	McLaren	British		2021-03-21	2024-07-24T14:
2	2	bmw_sauber	BMW Sauber	German		2021-03-21	2024-07-24T14:
3	3	williams	Williams	British		2021-03-21	2024-07-24T14:
4	4	renault	Renault	French		2021-03-21	2024-07-24T14:
5	5	toro_rosso	Toro Rosso	Italian		2021-03-21	2024-07-24T14:
6	6	ferrari	Ferrari	Italian		2021-03-21	2024-07-24T14:
7	7	toyota	Toyota	Japanese		2021-03-21	2024-07-24T14:
8	8	super_aguri	Super Aguri	Japanese		2021-03-21	2024-07-24T14:
9	9	red_bull	Red Bull	Austrian		2021-03-21	2024-07-24T14:
10	10	force_india	Force India	Indian		2021-03-21	2024-07-24T14:
11	11	honda	Honda	Japanese		2021-03-21	2024-07-24T14:
12	12	spyker	Spyker	Dutch		2021-03-21	2024-07-24T14:
13	13	mf1	MF1	Russian		2021-03-21	2024-07-24T14:
14	14	spyker_mf1	Spyker MF1	Dutch		2021-03-21	2024-07-24T14:

211 rows

```
dbutils.notebook.exit("Success")
```

```
Success
```

```
%python
html = """<h1 style="color:Black;text-align:center;font-family:Ariel">Report on Dominant Formula 1 Drivers </h1>"""
displayHTML(html)
```

Report on Dominant Formula 1 Drivers

```
SELECT driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points,
       RANK() OVER(ORDER BY AVG(calculated_points) DESC) driver_rank
  FROM f1_presentation.calculated_race_results
GROUP BY driver_name
HAVING COUNT(1) >= 50
ORDER BY avg_points DESC
```

Table

	driver_name	total_races	total_points	avg_points	driver_rank
1	Alain Prost	138	1156	8.376811594202898	1
2	Ayrton Senna	104	869	8.3557692307692 3	2
3	Jackie Stewart	61	507	8.311475409836065	3
4	Lewis Hamilton	232	1903	8.202586206896552	4
5	Michael Schumacher	226	1796	7.946902654867257	5
6	Nigel Mansell	90	713	7.92222222222222...	6
7	Niki Lauda	81	639	7.888888888888889	7
8	Sebastian Vettel	203	1565	7.70935960591133	8
9	Damon Hill	70	520	7.428571428571429	9
10	Juan Pablo Montoya	58	427	7.362068965517241	10
11	Nelson Piquet	115	845	7.34782608695652...	11
12	Mika Häkkinen	96	694	7.229166666666667	12
13	Jack Brabham	67	473	7.059701492537314	13
14	Bruce McLaren	58	404	6.96551724137931...	14
15	Denny Hulme	72	497	6.902777777777778	15

64 rows

```
CREATE OR REPLACE TEMP VIEW v_dominant_drivers
AS
SELECT driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points,
       RANK() OVER(ORDER BY AVG(calculated_points) DESC) driver_rank
  FROM f1_presentation.calculated_race_results
GROUP BY driver_name
HAVING COUNT(1) >= 50
ORDER BY avg_points DESC
```

OK

```
SELECT race_year,
       driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
GROUP BY race_year, driver_name
ORDER BY race_year, avg_points DESC
```

Table

	race_year	driver_name	total_races	total_points	avg_points
1	1950	Johnnie Parsons	1	10	10
2	1950	Juan Fangio	3	30	10
3	1950	Bill Holland	1	9	9
4	1950	Dorino Serafini	1	9	9
5	1950	Luigi Fagioli	5	44	8.8
6	1950	Nino Farina	5	41	8.2
7	1950	Alberto Ascari	3	24	8
8	1950	Mauri Rose	1	8	8
9	1950	Reg Parnell	1	8	8
10	1950	Robert Manzon	1	7	7
11	1950	Raymond Sommer	1	7	7
12	1950	Cecil Green	1	7	7
13	1950	Louis Rosier	5	34	6.8
14	1950	Prince Bira	2	13	6.5
15	1950	Tony Bettenhausen	1	6	6

1,971 rows

```
SELECT race_year,
       driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
 WHERE driver_name IN (SELECT driver_name FROM v_dominant_drivers WHERE driver_rank <= 10)
GROUP BY race_year, driver_name
ORDER BY race_year, avg_points DESC
```

Table | Visualization 1 | Data Profile 1

	race_year	driver_name	total_races	total_points	avg_points
1	1965	Jackie Stewart	7	56	8
2	1966	Jackie Stewart	3	23	7.666666666666667
3	1967	Jackie Stewart	2	17	8.5
4	1968	Jackie Stewart	8	59	7.375
5	1969	Jackie Stewart	8	76	9.5
6	1970	Jackie Stewart	5	38	7.6
7	1971	Jackie Stewart	8	75	9.375
8	1972	Jackie Stewart	7	60	8.571428571428571
9	1972	Niki Lauda	3	7	2.33333333333333...
10	1973	Jackie Stewart	13	103	7.923076923076923
11	1973	Niki Lauda	3	11	3.66666666666666...
12	1974	Niki Lauda	6	53	8.833333333333334
13	1975	Niki Lauda	13	100	7.69230769230769...
14	1976	Niki Lauda	11	94	8.545454545454545

14	1976	Niki Lauda	11	94	8.545454545454545
15	1977	Niki Lauda	12	105	8.75

118 rows

```
SELECT race_year,
       driver_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
 WHERE driver_name IN (SELECT driver_name FROM v_dominant_drivers WHERE driver_rank <= 10)
GROUP BY race_year, driver_name
ORDER BY race_year, avg_points DESC
```

Table | Visualization 1

	race_year	driver_name	total_races	total_points	avg_points
1	1965	Jackie Stewart	7	56	8
2	1966	Jackie Stewart	3	23	7.666666666666667
3	1967	Jackie Stewart	2	17	8.5
4	1968	Jackie Stewart	8	59	7.375
5	1969	Jackie Stewart	8	76	9.5
6	1970	Jackie Stewart	5	38	7.6
7	1971	Jackie Stewart	8	75	9.375
8	1972	Jackie Stewart	7	60	8.571428571428571
9	1972	Niki Lauda	3	7	2.33333333333333...
10	1973	Jackie Stewart	13	103	7.923076923076923
11	1973	Niki Lauda	3	11	3.66666666666666...
12	1974	Niki Lauda	6	53	8.833333333333334
13	1975	Niki Lauda	13	100	7.69230769230769...
14	1976	Niki Lauda	11	94	8.545454545454545
15	1977	Niki Lauda	12	105	8.75

118 rows

Table | Visualization 1

	race_year	driver_name	total_races	total_points	avg_points
1	1965	Jackie Stewart	7	56	8
2	1966	Jackie Stewart	3	23	7.666666666666667
3	1967	Jackie Stewart	2	17	8.5
4	1968	Jackie Stewart	8	59	7.375
5	1969	Jackie Stewart	8	76	9.5
6	1970	Jackie Stewart	5	38	7.6
7	1971	Jackie Stewart	8	75	9.375
8	1972	Jackie Stewart	7	60	8.571428571428571
9	1972	Niki Lauda	3	7	2.33333333333333...
10	1973	Jackie Stewart	13	103	7.923076923076923
11	1973	Niki Lauda	3	11	3.66666666666666...
12	1974	Niki Lauda	6	53	8.833333333333334
13	1975	Niki Lauda	13	100	7.69230769230769...
14	1976	Niki Lauda	11	94	8.545454545454545
15	1977	Niki Lauda	12	105	8.75

118 rows

databricks trans / 4.calculated_race_results

(https://databricks.com)

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
spark.sql(f"""
              CREATE TABLE IF NOT EXISTS f1_presentation.calculated_race_results
              (
              race_year INT,
              team_name STRING,
              driver_id INT,
              driver_name STRING,
              race_id INT,
              position INT,
              points INT,
              calculated_points INT,
              created_date TIMESTAMP,
              updated_date TIMESTAMP
              )
              USING DELTA
""")
```

```
DataFrame[]
```

```
# %sql
# -- use f1_processed;
```

```

```

```
# -- select
# --     races.race_year,
# --     constructors.name as team_name,
# --     drivers.name as driver_name,
# --     results.position,
# --     results.points,
# --     11 - results.position as calculated_points
# --   from results
# --   join drivers on (drivers.driver_id = results.driver_id)
# --   join constructors on (constructors.constructor_id = results.constructor_id)
# --   join races on (races.race_id = results.race_id)
# --   where  results.position <=10  ;
```

```

```

```
# %sql
# -- create table f1_presentation.calculated_race_results
# -- using parquet
# -- as
# -- select
# --     races.race_year,
# --     constructors.name as team_name,
# --     drivers.name as driver_name,
# --     results.position,
# --     results.points,
# --     11 - results.position as calculated_points
# --   from f1_processed.results
# --   join f1_processed.drivers on (drivers.driver_id = results.driver_id)
# --   join f1_processed.constructors on (constructors.constructor_id = results.constructor_id)
# --   join f1_processed.races on (races.race_id = results.race_id)
# --   where  results.position <=10  ;
```

```
# %sql
# -- select * from f1_presentation.calculated_race_results;
```

```
spark.sql(f"""
              CREATE OR REPLACE TEMP VIEW race_result_updated
              AS
              SELECT races.race_year,
                     constructors.name AS team_name,
                     drivers.driver_id,
                     drivers.name AS driver_name,
                     races.race_id,
                     results.position,
                     results.points,
                     11 - results.position AS calculated_points
                FROM f1_processed.results
                JOIN f1_processed.drivers ON (results.driver_id = drivers.driver_id)
                JOIN f1_processed.constructors ON (results.constructor_id = constructors.constructor_id)
                JOIN f1_processed.races ON (results.race_id = races.race_id)
               WHERE results.position <= 10
                 AND results.file_date = '{v_file_date}'
""")
```

```
DataFrame[]
```

```
# %sql select * from race_result_updated where race_year = 2021;
```

Table

	race_year	team_name	driver_id	driver_name	race_id	position	points	ca
1	2021	Mercedes	1	Lewis Hamilton	1053	2	19	
2	2021	Alpine F1 Team	4	Fernando Alonso	1053	10	1	
3	2021	AlphaTauri	842	Pierre Gasly	1053	7	6	
4	2021	McLaren	817	Daniel Ricciardo	1053	6	8	
5	2021	Red Bull	830	Max Verstappen	1053	1	25	
6	2021	Ferrari	832	Carlos Sainz	1053	5	10	
7	2021	Alpine F1 Team	839	Esteban Ocon	1053	9	2	
8	2021	Aston Martin	840	Lance Stroll	1053	8	4	
9	2021	Ferrari	844	Charles Leclerc	1053	4	12	
10	2021	McLaren	846	Lando Norris	1053	3	15	

(https://databricks.com)

```
display(dbutils.fs.ls('/'))
```

Table

	path	name	size	modificationTime
1	dbfs:/FileStore/	FileStore/	0	1721486034000
2	dbfs:/Volume/	Volume/	0	0
3	dbfs:/Volumes/	Volumes/	0	0
4	dbfs:/databricks-dataset...	databricks-dataset...	0	0
5	dbfs:/databricks-results/	databricks-results/	0	0
6	dbfs:/volume/	volume/	0	0
7	dbfs:/volumes/	volumes/	0	0

7 rows

```
display(dbutils.fs.ls('/FileStore'))
```

Table

	path	name	size	modificationTime
1	dbfs:/FileStore/circuits.c...	circuits.csv	10044	1721486073000
2	dbfs:/FileStore/tables/	tables/	0	1721486034000

2 rows

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7
1	circuitId	circuitRef	name	location	country	lat	lng	alt
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18
4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109
6	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.405	130
7	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
8	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228	13
9	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361	228
10	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153
11	10	hockenheimri...	Hockenheimring	Hockenheim	Germany	49.3278	8.56583	103
12	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.2486	264
13	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667	4
14	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139	401

78 rows

Step 1 - Read the JSON file using the spark dataframe reader API

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType
```

```
name_schema = StructType(fields=[StructField("forename", StringType(), True),
                                 StructField("surname", StringType(), True)

])
```

```
drivers_schema = StructType(fields=[StructField("driverId", IntegerType(), False),
                                    StructField("driverRef", StringType(), True),
                                    StructField("number", IntegerType(), True),
                                    StructField("code", StringType(), True),
                                    StructField("name", name_schema),
                                    StructField("dob", DateType(), True),
                                    StructField("nationality", StringType(), True),
                                    StructField("url", StringType(), True)
])
```

```
drivers_df = spark.read \
.schema(drivers_schema) \
.json(f"{raw_folder_path}/{v_file_date}/drivers.json")
```

```
display(drivers_df)
```



	driverId	driverRef	number	code	name	dob	na
1	1	hamilton	44	HAM	{"forename":"Lewis","surname":"Hamilton"}	1985-01-07	British
2	2	heidfeld	null	HEI	{"forename":"Nick","surname":"Heidfeld"}	1977-05-10	Germa
3	3	rosberg	6	ROS	{"forename":"Nico","surname":"Rosberg"}	1985-06-27	Germa
4	4	alonso	14	ALO	{"forename":"Fernando","surname":"Alonso"}	1981-07-29	Spanis
5	5	kovalainen	null	KOV	{"forename":"Heikki","surname":"Kovalainen"}	1981-10-19	Finnish
6	6	nakajima	null	NAK	{"forename":"Kazuki","surname":"Nakajima"}	1985-01-11	Japane
7	7	bourdais	null	BOU	{"forename":"Sébastien","surname":"Bourdais"}	1979-02-28	Frenc
8	8	raikkonen	7	RAI	{"forename":"Kimi","surname":"Räikkönen"}	1979-10-17	Finnish
9	9	kubica	88	KUB	{"forename":"Robert","surname":"Kubica"}	1984-12-07	Polish
10	10	glock	null	GLO	{"forename":"Timo","surname":"Glock"}	1982-03-18	Germa
11	11	sato	null	SAT	{"forename":"Takuma","surname":"Sato"}	1977-01-28	Japane
12	12	piquet_jr	null	PIQ	{"forename":"Nelson","surname":"Piquet Jr."}	1985-07-25	Brazili
13	13	massa	19	MAS	{"forename":"Felipe","surname":"Massa"}	1981-04-25	Brazili
14	14	coulthard	null	COU	{"forename":"David","surname":"Coulthard"}	1971-03-27	British

853 rows

Step 2 - Rename columns and add new columns

- driverId renamed to driver_id
- driverRef renamed to driver_ref
- ingestion date added
- name added with concatenation of forename and surname

```
from pyspark.sql.functions import col, concat, current_timestamp, lit
```

```
drivers_with_ingestion_date_df = add_ingestion_date(drivers_df)
```

```
drivers_with_columns_df = drivers_df.withColumnRenamed("driverId", "driver_id") \
                                    .withColumnRenamed("driverRef", "driver_ref") \
                                    .withColumn("ingestion_date", current_timestamp()) \
                                    .withColumn("name", concat(col("name.forename"), lit(" "), col("name.surname"))) \
                                    .withColumn("data_source", lit(v_data_source)) \
                                    .withColumn("file_date", lit(v_file_date))
```

```
display(drivers_with_columns_df)
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	url
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	http://en
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	http://en
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	http://en
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	http://en
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	http://en
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	http://en
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	http://en
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	http://en
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	http://en

10	10	glock	null	GLO	Timo Glock	1982-03-18	German	http://en
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	http://en
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	http://en
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	http://en
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	http://en

853 rows

Step 3 - Drop the unwanted columns

- name.forename
- name.surname
- url

```
drivers_final_df = drivers_with_columns_df.drop(col("url"))
```

```
display(drivers_final_df)
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	2024-07

853 rows

Step 4 - Write to output to processed container in parquet format

```
# drivers_final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.drivers")
```

```
drivers_final_df.write.mode("overwrite").format("delta").saveAsTable("f1_processed.drivers")
```

```
# display(spark.read.parquet(f"{processed_folder_path}/drivers"))
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	2024-07

853 rows

```
%fs
ls /mnt/formula1dl1204/processed/drivers
```

Table

	path	name
1	dbfs:/mnt/formula1dl1204/processed/drivers/_delta_log/	_delta_log/
2	dbfs:/mnt/formula1dl1204/processed/drivers/part-00000-f9b542a1-1a36-4b0b-b616-45e82dae5518-c000.snappy.parqu...	part-00000-f9b542a1-1a36-4b0b-b616-45

2 rows

```
%sql
select * from f1_processed.drivers;
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07

Access dataframes using sql

Objectives

1. Create temporary views on dataframes
2. Access the view from SQL cell
3. Access the view from Python cell

```
%run "../includes/configuration"
```

```
race_results_df = spark.read.parquet(f"{presentation_folder_path}/race_results")
```

Local Temporary View

```
race_results_df.createOrReplaceTempView("v_race_results")
```

```
%sql
select *
from v_race_results
where race_year = 2020
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Lewis Hamilton	British
2	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Max Verstappen	Dutch
3	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Alexander Albon	Thai
4	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Lando Norris	British
5	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Carlos Sainz	Spanish
6	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Pierre Gasly	French
7	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Daniel Ricciardo	Australian
8	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Valtteri Bottas	Finnish
9	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Esteban Ocon	French
10	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Charles Leclerc	Monegasque
11	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Daniil Kvyat	Russian
12	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	George Russell	British
13	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Sebastian Vettel	German
14	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Nicholas Latifi	Canadian

340 rows

```
p_race_year = 2019
```

```
race_results_2019_df = spark.sql(f"select * from v_race_results where race_year = {p_race_year}")
```

```
race_results_2019_df = spark.sql("select * from v_race_results where race_year = 2019")
```

```
display(race_results_2019_df)
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Lewis Hamilton	British
2	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Max Verstappen	Dutch
3	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Sebastian Vettel	German
4	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Charles Leclerc	Monegasque
5	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Carlos Sainz	Spanish
6	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Pierre Gasly	French
7	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Kimi Räikkönen	Finnish
8	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Valtteri Bottas	Finnish
9	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Lando Norris	British
10	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Alexander Albon	Thai
11	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Sergio Pérez	Mexican
12	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Nico Hülkenberg	German
13	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Kevin Magnussen	Danish
14	2019	Hungarian Grand Prix	2019-08-04T13:10:00.000+00:...	Budapest	Daniel Ricciardo	Australian

420 rows

Global Temporary View

Objectives

1. Create Global views on dataframes
2. Access the view from SQL cell
3. Access the view from Python cell
4. Access the view from another Notebook

```
race_results_df.createOrReplaceGlobalTempView("gv_race_results")
```

```
%sql
show tables
```

Table

	database	tableName	isTemporary
1		v_race_results	true

1 row

```
%sql
show tables in global_temp
```

Table

	database	tableName	isTemporary
1	global_temp	gv_race_results	true
2		v_race_results	true

2 rows

```
%sql
select * from global_temp.gv_race_results
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Jenson Button	British
2	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Rubens Barrichello	Brazilian
3	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Mark Webber	Australian
4	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Sebastian Vettel	German
5	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Fernando Alonso	Spanish
6	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Felipe Massa	Brazilian
7	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nick Heidfeld	German
8	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nico Rosberg	German
9	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Lewis Hamilton	British
10	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Timo Glock	German
11	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Robert Kubica	Polish
12	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nelson Piquet Jr.	Brazilian
13	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Kazuki Nakajima	Japanese
14	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Giancarlo Fisichella	Italian

10,000+ rows | Truncated data due to row limit

```
spark.sql("select * from global_temp.gv_race_results").show()
```

```
-------+-----------+------+--------+--------------------+
|     2009|Spanish Grand Prix|2009-05-10 12:00:00|       Montmeló|       Jenson Button|           British|      Brawn|   1|
17|1:37:19.202|  10.0|       1|2024-07-21 19:41:...|
|     2009|Spanish Grand Prix|2009-05-10 12:00:00|       Montmeló|  Rubens Barrichello|         Brazilian|      Brawn|   3|
28|   +13.056|   8.0|       2|2024-07-21 19:41:...|
|     2009|Spanish Grand Prix|2009-05-10 12:00:00|       Montmeló|         Mark Webber|        Australian|   Red Bull|   5|
18|   +13.924|   6.0|       3|2024-07-21 19:41:...|
|     2009|Spanish Grand Prix|2009-05-10 12:00:00|       Montmeló|    Sebastian Vettel|            German|   Red Bull|   2|
41|   +18.941|   5.0|       4|2024-07-21 19:41:...|
```

```
%python
html = """<h1 style="color:Black;text-align:center;font-family:Ariel">Report on Dominant Formula 1 Teams </h1>"""
displayHTML(html)
```

Report on Dominant Formula 1 Teams

```
CREATE OR REPLACE TEMP VIEW v_dominant_teams
AS
SELECT team_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points,
       RANK() OVER(ORDER BY AVG(calculated_points) DESC) team_rank
  FROM f1_presentation.calculated_race_results
GROUP BY team_name
HAVING COUNT(1) >= 100
ORDER BY avg_points DESC
```

OK

```
SELECT * FROM v_dominant_teams;
```

Table

	team_name	total_races	total_points	avg_points	team_rank
1	Mercedes	385	2931	7.612987012987013	1
2	Ferrari	1478	10500	7.104194857916103	2
3	McLaren	1032	6865	6.65213178294573...	3
4	Red Bull	433	2851	6.584295612009238	4
5	Williams	745	4759	6.387919463087249	5
6	Benetton	296	1806	6.101351351351352	6
7	Team Lotus	352	2100	5.965909090909091	7
8	Cooper-Climax	130	772	5.938461538461539	8
9	Brabham	235	1373	5.842553191489362	9
10	BRM	215	1212	5.637209302325582	10
11	Renault	385	2105	5.467532467532467	11
12	Maserati	179	950	5.307262569832402	12
13	Tyrrell	339	1749	5.15929203539823	13
14	Jordan	184	882	4.793478260869565	14
15	Ligier	248	1134	4.57258064516129	15

23 rows

```
SELECT race_year,
       team_name,
       COUNT(1) AS total_races,
       SUM(calculated_points) AS total_points,
       AVG(calculated_points) AS avg_points
  FROM f1_presentation.calculated_race_results
 WHERE team_name IN (SELECT team_name FROM v_dominant_teams WHERE team_rank <= 5)
GROUP BY race_year, team_name
ORDER BY race_year, avg_points DESC
```

Table | Visualization 1

	race_year	team_name	total_races	total_points	avg_points
1	1950	Ferrari	8	57	7.125
2	1951	Ferrari	22	168	7.636363636363637
3	1952	Ferrari	28	202	7.214285714285714
4	1953	Ferrari	35	230	6.571428571428571
5	1954	Mercedes	12	96	8
6	1954	Ferrari	25	175	7
7	1955	Mercedes	15	124	8.266666666666667
8	1955	Ferrari	21	131	6.238095238095238
9	1956	Ferrari	23	172	7.478260869565218
10	1957	Ferrari	20	130	6.5
11	1958	Ferrari	21	159	7.571428571428571
12	1959	Ferrari	19	132	6.947368421052632
13	1960	Ferrari	17	102	6
14	1961	Ferrari	19	152	8
15	1962	Ferrari	12	73	6.083333333333333

194 rows

Table | Visualization 1

	race_year	team_name	total_races	total_points	avg_points
48	1980	Ferrari	12	47	3.9166666666666...
49	1981	Williams	21	154	7.333333333333333
50	1981	Ferrari	13	72	5.538461538461538
51	1981	McLaren	13	70	5.384615384615385
52	1982	Ferrari	16	122	7.625
53	1982	McLaren	17	118	6.94117647058823...
54	1982	Williams	20	128	6.4
55	1983	Ferrari	20	148	7.4
56	1983	McLaren	12	74	6.166666666666667
57	1983	Williams	16	94	5.875
58	1984	McLaren	21	191	9.095238095238095
59	1984	Ferrari	18	123	6.833333333333333
60	1984	Williams	10	60	6
61	1985	McLaren	16	134	8.375
62	1985	Ferrari	20	147	7.35
63	1985	Williams	17	123	7.235294117647059

194 rows

Step 1 - Read the JSON file using the spark dataframe reader API

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-28")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, FloatType
```

```
results_schema = StructType(fields=[StructField("resultId", IntegerType(), False),
                                    StructField("raceId", IntegerType(), True),
                                    StructField("driverId", IntegerType(), True),
                                    StructField("constructorId", IntegerType(), True),
                                    StructField("number", IntegerType(), True),
                                    StructField("grid", IntegerType(), True),
                                    StructField("position", IntegerType(), True),
                                    StructField("positionText", StringType(), True),
                                    StructField("positionOrder", IntegerType(), True),
                                    StructField("points", FloatType(), True),
                                    StructField("laps", IntegerType(), True),
                                    StructField("time", StringType(), True),
                                    StructField("milliseconds", IntegerType(), True),
                                    StructField("fastestLap", IntegerType(), True),
                                    StructField("rank", IntegerType(), True),
                                    StructField("fastestLapTime", StringType(), True),
                                    StructField("fastestLapSpeed", FloatType(), True),
                                    StructField("statusId", StringType(), True)])
```

```
results_df = spark.read \
.schema(results_schema) \
.json(f"{raw_folder_path}/{v_file_date}/results.json")
```

```
display(results_df)
```

Table



	resultId	raceId	driverId	constructorId	number	grid	position	positionTe
1	1	18	1	1	22	1	1	1
2	2	18	2	2	3	5	2	2
3	3	18	3	3	7	7	3	3
4	4	18	4	4	5	11	4	4
5	5	18	5	1	23	3	5	5
6	6	18	6	3	8	13	6	6
7	7	18	7	5	14	17	7	7
8	8	18	8	6	1	15	8	8
9	9	18	9	2	4	2	null	R
10	10	18	10	7	12	18	null	R
11	11	18	11	8	18	19	null	R
12	12	18	12	4	6	20	null	R
13	13	18	13	6	2	4	null	R
14	14	18	14	9	9	8	null	R

10,000+ rows | Truncated data due to row limit

Step 2 - Rename columns and add new columns

```
from pyspark.sql.functions import current_timestamp, lit
```

```
results_with_columns_df = results_df.withColumnRenamed("resultId", "result_id") \
                                    .withColumnRenamed("raceId", "race_id") \
                                    .withColumnRenamed("driverId", "driver_id") \
                                    .withColumnRenamed("constructorId", "constructor_id") \
                                    .withColumnRenamed("positionText", "position_text") \
                                    .withColumnRenamed("positionOrder", "position_order") \
                                    .withColumnRenamed("fastestLap", "fastest_lap") \
                                    .withColumnRenamed("fastestLapTime", "fastest_lap_time") \
                                    .withColumnRenamed("fastestLapSpeed", "fastest_lap_speed") \
                                    .withColumn("data_source", lit(v_data_source)) \
                                    .withColumn("file_date", lit(v_file_date))
```

```
results_with_ingestion_date_df = add_ingestion_date(results_with_columns_df)
```

```
display(results_with_columns_df)
```

Table

	result_id	race_id	driver_id	constructor_id	number	grid	position	positio
1	1	18	1	1	22	1	1	1
2	2	18	2	2	3	5	2	2
3	3	18	3	3	7	7	3	3
4	4	18	4	4	5	11	4	4
5	5	18	5	1	23	3	5	5
6	6	18	6	3	8	13	6	6
7	7	18	7	5	14	17	7	7
8	8	18	8	6	1	15	8	8
9	9	18	9	2	4	2	null	R

10	10	18	10	7	12	18	null	R
11	11	18	11	8	18	19	null	R
12	12	18	12	4	6	20	null	R
13	13	18	13	6	2	4	null	R
14	14	18	14	9	9	8	null	R

10,000+ rows | Truncated data due to row limit

Step 3 - Drop the unwanted column

```
from pyspark.sql.functions import col
```

```
results_final_df = results_with_columns_df.drop(col("statusId"))
```

```
display(results_final_df)
```

Table

	result_id	race_id	driver_id	constructor_id	number	grid	position	positio
1	1	18	1	1	22	1	1	1
2	2	18	2	2	3	5	2	2
3	3	18	3	3	7	7	3	3
4	4	18	4	4	5	11	4	4
5	5	18	5	1	23	3	5	5
6	6	18	6	3	8	13	6	6
7	7	18	7	5	14	17	7	7
8	8	18	8	6	1	15	8	8
9	9	18	9	2	4	2	null	R
10	10	18	10	7	12	18	null	R
11	11	18	11	8	18	19	null	R
12	12	18	12	4	6	20	null	R
13	13	18	13	6	2	4	null	R
14	14	18	14	9	9	8	null	R

10,000+ rows | Truncated data due to row limit

Step 4 - Write to output to processed container in parquet format

Method 1

```
# for race_id_list in results_final_df.select("race_id").distinct().collect():
#    if (spark._jsparkSession.catalog().tableExists("f1_processed.results")):
#      spark.sql(f"ALTER TABLE f1_processed.results DROP IF EXISTS PARTITION (race_id = {race_id_list.race_id})")
```

```
#results_final_df.write.mode("overwrite").partitionBy('race_id').format("parquet").saveAsTable("f1_processed.results")
```

Method 2

```
# overwrite_partition(results_final_df, 'f1_processed', 'results', 'race_id')
```

```
results_deduped_df = results_final_df.dropDuplicates(['race_id', 'driver_id'])
```

```
merge_condition = "tgt.result_id = src.result_id AND tgt.race_id = src.race_id"
merge_delta_data(results_deduped_df, 'f1_processed', 'results', processed_folder_path, merge_condition, 'race_id')
```

```
%sql
SELECT race_id, COUNT(1)
FROM f1_processed.results
GROUP BY race_id
ORDER BY race_id DESC;
```

Table

	race_id	count(1)
1	1053	20
2	1052	20
3	1047	20
4	1046	20
5	1045	20
6	1044	20
7	1043	20
8	1042	20
9	1041	20
10	1040	20
11	1039	20
12	1038	20
13	1037	20
14	1036	20
15	1035	20

1,037 rows

```
%sql
select race_id, driver_id, count(1)
from f1_processed.results
group by race_id, driver_id
having count(1) > 1
order by race_id, driver_id desc;
```

Table

	race_id	driver_id	count(1)

1	540	229	2
2	717	373	2
3	733	465	2
4	742	475	2
5	745	418	2
6	746	475	2
7	766	541	2
8	770	479	2
9	774	566	2
10	776	581	2
11	777	578	2
12	777	501	2
13	779	356	2
14	780	581	2
15	780	479	2

85 rows

```
# display(spark.read.parquet(f"{processed_folder_path}/results"))
```

```
%sql
select * from f1_processed.results;
```

Table

	result_id	race_id	driver_id	constructor_id	number	grid	position	positio
1	20327	337	3	131	4	5	5	5
2	20337	337	5	205	19	21	15	15
3	20333	337	9	4	11	9	11	11
4	20334	337	16	10	14	10	12	12
5	20329	337	18	1	1	8	7	7
6	20326	337	20	9	5	1	4	4
7	20328	337	30	131	3	7	6	6
8	20336	337	807	3	10	13	14	14
9	20345	337	810	166	25	22	null	R
10	20341	337	811	164	21	23	null	R
11	20325	337	1	1	2	4	3	3
12	20323	337	4	6	8	3	1	1
13	20342	337	10	166	24	19	null	R
14	20324	337	13	6	7	2	2	2

10,000+ rows | Truncated data due to row limit

Success

databricks Set-up / 5.mount_adls_using_service_principal

(https://databricks.com)

```
client_id = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-id')
tenant_id = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-tenant-id')
client_secret = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-secret')
```

```
configs = {"fs.azure.account.auth.type": "OAuth",
          "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
          "fs.azure.account.oauth2.client.id": client_id,
          "fs.azure.account.oauth2.client.secret": client_secret,
          "fs.azure.account.oauth2.client.endpoint": f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"}
```

```
dbutils.fs.mount(
  source = "abfss://demo@formula1dl1204.dfs.core.windows.net/",
  mount_point = "/mnt/formula1dl204/demo",
  extra_configs = configs)
```

```
True
```

```
display(dbutils.fs.mounts())
```

Table

	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.n...	
3	/Volumes	UnityCatalogVolumes	
4	/databricks/mlflow-tracki...	databricks/mlflow-tracking	
5	/databricks-results	databricks-results	
6	/databricks/mlflow-registry	databricks/mlflow-registry	
7	/Volume	DbfsReserved	
8	/volumes	DbfsReserved	
9	/	DatabricksRoot	
10	/volume	DbfsReserved	

10 rows

```
display(dbutils.fs.ls("/mnt"))
```

Table

	path	name	size	modificationTime
1	dbfs:/mnt/formula1dl/	formula1dl/	0	1721488475000
2	dbfs:/mnt/formula1dl20...	formula1dl204/	0	1721488522000

2 rows

```
display(dbutils.fs.ls("/mnt/formula1dl204/demo"))
```

Table

	path	name	size	modificationTime
1	dbfs:/mnt/formula1dl204/demo/circuits.c...	circuits.csv	10044	1721470200000

1 row

```
display(spark.read.csv("/mnt/formula1dl204/demo/circuits.csv"))
```

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7
1	circuitId	circuitRef	name	location	country	lat	lng	alt
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18
4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109
6	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.405	130
7	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
8	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228	13
9	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361	228
10	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153
11	10	hockenheimri...	Hockenheimring	Hockenheim	Germany	49.3278	8.56583	103
12	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.2486	264
13	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667	4
14	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139	401

78 rows

```
display(dbutils.fs.ls("abfss://demo@formula1dl1204.dfs.core.windows.net"))
```

Table

	path	name	size	modificationTime
1	abfss://demo@formula1dl1204.dfs.core.windows.net/circuits.c...	circuits.csv	10044	1721470200000

1 row

databricks demo / 5.sql_temp_global_view_demo

(https://databricks.com)

```
1  %sql
2  select * from v_race_results
```

```
> [TABLE_OR_VIEW_NOT_FOUND (https://docs.databricks.com/error-messages/error-classes.html#table_or_view_not_found)] The table or view `v_race_results` cannot be found. Verify the spelling and correctness of the schema and catalog.
If you did not qualify the name with a schema, verify the current_schema() output, or qualify the name with the correct schema an…
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Jenson Button	British
2	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Rubens Barrichello	Brazilian
3	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Mark Webber	Australian
4	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Sebastian Vettel	German
5	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Fernando Alonso	Spanish
6	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Felipe Massa	Brazilian
7	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nick Heidfeld	German
8	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nico Rosberg	German
9	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Lewis Hamilton	British
10	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Timo Glock	German
11	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Robert Kubica	Polish
12	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nelson Piquet Jr.	Brazilian
13	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Kazuki Nakajima	Japanese
14	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Giancarlo Fisichella	Italian

10,000+ rows | Truncated data due to row limit

Ingest pit_stops.json file

Step 1 - Read the JSON file using the spark dataframe reader API

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-28")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType
```

```
pit_stops_schema = StructType(fields=[StructField("raceId", IntegerType(), False),
                                      StructField("driverId", IntegerType(), True),
                                      StructField("stop", StringType(), True),
                                      StructField("lap", IntegerType(), True),
                                      StructField("time", StringType(), True),
                                      StructField("duration", StringType(), True),
                                      StructField("milliseconds", IntegerType(), True)
                                     ])
```

```
pit_stops_df = spark.read \
.schema(pit_stops_schema) \
.option("multiLine", True) \
.json(f"{raw_folder_path}/{v_file_date}/pit_stops.json")
```

```
# pit_stops_df = spark.read \
# .option("multiLine", True) \
# .json(f"{raw_folder_path}/pit_stops.json")
```

```
display(pit_stops_df)
```

Table

	raceId	driverId	stop	lap	time	duration	milliseconds
1	1053	839	1	1	15:05:16	30.866	30866
2	1053	20	1	3	15:10:09	32.024	32024
3	1053	854	1	5	15:15:11	51.007	51007
4	1053	853	1	12	15:27:20	31.168	31168
5	1053	842	1	14	15:30:10	31.068	31068
6	1053	20	2	20	15:39:11	31.184	31184
7	1053	854	2	21	15:41:24	32.479	32479
8	1053	20	3	22	15:42:52	39.502	39502
9	1053	853	2	23	15:45:20	31.5	31500
10	1053	852	1	25	15:46:39	30.696	30696
11	1053	847	1	26	15:47:53	29.983	29983
12	1053	8	1	26	15:47:55	30.28	30280
13	1053	830	1	27	15:47:59	29.809	29809
14	1053	842	2	26	15:48:45	32.277	32277
15	1053	832	1	27	15:48:51	30.856	30856

56 rows

Step 2 - Rename columns and add new columns

- Rename driverId and raceId
- Add ingestion_date with current timestamp

```
pit_stops_with_ingestion_date_df = add_ingestion_date(pit_stops_df)
```

```
from pyspark.sql.functions import current_timestamp, lit
```

```
final_df = pit_stops_df.withColumnRenamed("driverId", "driver_id") \
.withColumnRenamed("raceId", "race_id") \
.withColumn("ingestion_date", current_timestamp()) \
.withColumn("data_source", lit(v_data_source)) \
.withColumn("file_date", lit(v_file_date))
```

Step 3 - Write to output to processed container in parquet format

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.pit_stops")
```

```
# overwrite_partition(final_df, 'f1_processed', 'pit_stops', 'race_id')
```

```
merge_condition = "tgt.race_id = src.race_id AND tgt.driver_id = src.driver_id AND tgt.stop = src.stop AND tgt.race_id = src.race_i
merge_delta_data(final_df, 'f1_processed', 'pit_stops', processed_folder_path, merge_condition, 'race_id')
```

```
display(final_df)
```

Table

	race_id	driver_id	stop	lap	time	duration	milliseconds	ingestion_date
1	1053	839	1	1	15:05:16	30.866	30866	2024-07-24T13:48:32
2	1053	20	1	3	15:10:09	32.024	32024	2024-07-24T13:48:32
3	1053	854	1	5	15:15:11	51.007	51007	2024-07-24T13:48:32
4	1053	853	1	12	15:27:20	31.168	31168	2024-07-24T13:48:32
5	1053	842	1	14	15:30:10	31.068	31068	2024-07-24T13:48:32
6	1053	20	2	20	15:39:11	31.184	31184	2024-07-24T13:48:32
7	1053	854	2	21	15:41:24	32.479	32479	2024-07-24T13:48:32
8	1053	20	3	22	15:42:52	39.502	39502	2024-07-24T13:48:32
9	1053	853	2	23	15:45:20	31.5	31500	2024-07-24T13:48:32
10	1053	852	1	25	15:46:39	30.696	30696	2024-07-24T13:48:32
11	1053	847	1	26	15:47:53	29.983	29983	2024-07-24T13:48:32
12	1053	8	1	26	15:47:55	30.28	30280	2024-07-24T13:48:32
13	1053	830	1	27	15:47:59	29.809	29809	2024-07-24T13:48:32
14	1053	842	2	26	15:48:45	32.277	32277	2024-07-24T13:48:32

56 rows

```
%sql
select * from f1_processed.pit_stops;
```

Table

	race_id	driver_id	stop	lap	time	duration	milliseconds	ingestion_date
1	936	833	1	6	14:15:43	26.392	26392	2024-07-24T13:47:14
2	936	822	1	13	14:25:47	22.974	22974	2024-07-24T13:47:14
3	936	826	1	13	14:25:51	22.072	22072	2024-07-24T13:47:14
4	936	13	1	14	14:27:28	27.936	27936	2024-07-24T13:47:14
5	936	830	1	14	14:27:30	22.536	22536	2024-07-24T13:47:14
6	936	18	1	14	14:27:33	22.231	22231	2024-07-24T13:47:14
7	936	154	1	14	14:27:34	22.805	22805	2024-07-24T13:47:14
8	936	831	1	14	14:27:41	22.729	22729	2024-07-24T13:47:14
9	936	807	1	15	14:28:49	22.573	22573	2024-07-24T13:47:14
10	936	832	1	15	14:28:58	25.096	25096	2024-07-24T13:47:14
11	936	4	1	15	14:29:00	21.567	21567	2024-07-24T13:47:14
12	936	815	1	16	14:30:25	22.068	22068	2024-07-24T13:47:14
13	936	828	1	16	14:30:36	23.005	23005	2024-07-24T13:47:14
14	936	1	1	19	14:34:42	23.146	23146	2024-07-24T13:47:14

8,126 rows

```
Success
```

databricks Set-up / 6.mount_adls_container_for_project

(https://databricks.com)

```
def mount_adls(storage_account_name, container_name):
    # Get secrets from Key Vault
    client_id = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-id')
    tenant_id = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-tenant-id')
    client_secret = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-secret')

    # Set spark configurations
    configs = {"fs.azure.account.auth.type": "OAuth",
          "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
          "fs.azure.account.oauth2.client.id": client_id,
          "fs.azure.account.oauth2.client.secret": client_secret,
          "fs.azure.account.oauth2.client.endpoint": f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"}

    # Unmount the mount point if it already exists
    if any(mount.mountPoint == f"/mnt/{storage_account_name}/{container_name}" for mount in dbutils.fs.mounts()):
        dbutils.fs.unmount(f"/mnt/{storage_account_name}/{container_name}")

    # Mount the storage account container
    dbutils.fs.mount(
      source = f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/",
      mount_point = f"/mnt/{storage_account_name}/{container_name}",
      extra_configs = configs)

    display(dbutils.fs.mounts())
```

```
##### Mount Raw Container

mount_adls('formula1dl1204', 'raw')
```

Table

	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.n...	
3	/Volumes	UnityCatalogVolumes	
4	/mnt/formula1dl1204/raw	abfss://raw@formula1dl1204.dfs.core.windows.net/	
5	/databricks/mlflow-tracki...	databricks/mlflow-tracking	
6	/databricks-results	databricks-results	
7	/databricks/mlflow-registry	databricks/mlflow-registry	
8	/Volume	DbfsReserved	
9	/volumes	DbfsReserved	
10	/	DatabricksRoot	
11	/volume	DbfsReserved	

11 rows

```
##### Mount processed Container

mount_adls('formula1dl1204', 'processed')
```

Table

	mountPoint	source	encryptionType

1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.net/	
3	/mnt/formula1dl1204/process	abfss://processed@formula1dl1204.dfs.core.windows.n	
4	/Volumes	UnityCatalogVolumes	
5	/mnt/formula1dl1204/raw	abfss://raw@formula1dl1204.dfs.core.windows.net/	
6	/databricks/mlflow-tracking	databricks/mlflow-tracking	
7	/databricks-results	databricks-results	
8	/databricks/mlflow-registry	databricks/mlflow-registry	
9	/Volume	DbfsReserved	
10	/volumes	DbfsReserved	
11	/	DatabricksRoot	
12	/volume	DbfsReserved	

12 rows

```
##### Mount presentation Container
mount_adls('formula1dl1204', 'presentation')
```

Table

	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl1204/presentati...	abfss://presentation@formula1dl1204.dfs.core.windows.n...	
3	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.net/	
4	/mnt/formula1dl1204/processed	abfss://processed@formula1dl1204.dfs.core.windows.net/	
5	/Volumes	UnityCatalogVolumes	
6	/mnt/formula1dl1204/raw	abfss://raw@formula1dl1204.dfs.core.windows.net/	
7	/databricks/mlflow-tracking	databricks/mlflow-tracking	
8	/databricks-results	databricks-results	
9	/databricks/mlflow-registry	databricks/mlflow-registry	
10	/Volume	DbfsReserved	
11	/volumes	DbfsReserved	
12	/	DatabricksRoot	
13	/volume	DbfsReserved	

13 rows

```
client_id = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-id')
tenant_id = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-tenant-id')
client_secret = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-secret')
```

```
configs = {"fs.azure.account.auth.type": "OAuth",
          "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
          "fs.azure.account.oauth2.client.id": client_id,
          "fs.azure.account.oauth2.client.secret": client_secret,
          "fs.azure.account.oauth2.client.endpoint": f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"}
```

```
dbutils.fs.mount(
  source = "abfss://demo@formula1dl1204.dfs.core.windows.net/",
  mount_point = "/mnt/formula1dl204/demo",
  extra_configs = configs)
```

```
True
```

```
display(dbutils.fs.mounts())
```

Table

	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/formula1dl204/demo	abfss://demo@formula1dl1204.dfs.core.windows.n...	
3	/Volumes	UnityCatalogVolumes	
4	/databricks/mlflow-tracki...	databricks/mlflow-tracking	
5	/databricks-results	databricks-results	
6	/databricks/mlflow-registry	databricks/mlflow-registry	
7	/Volume	DbfsReserved	
8	/volumes	DbfsReserved	
9	/	DatabricksRoot	
10	/volume	DbfsReserved	

10 rows

```
display(dbutils.fs.ls("/mnt"))
```

Table

	path	name	size	modificationTime
1	dbfs:/mnt/formula1dl/	formula1dl/	0	1721488475000
2	dbfs:/mnt/formula1dl20...	formula1dl204/	0	1721488522000

2 rows

```
display(dbutils.fs.ls("/mnt/formula1dl204/demo"))
```

Table

	path	name	size	modificationTime
1	dbfs:/mnt/formula1dl204/demo/circuits.c...	circuits.csv	10044	1721470200000

1 row

```
display(spark.read.csv("/mnt/formula1dl204/demo/circuits.csv"))
```

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7
1	circuitId	circuitRef	name	location	country	lat	lng	alt
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18

4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109
6	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.405	130
7	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
8	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228	13
9	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361	228
10	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153
11	10	hockenheimri...	Hockenheimring	Hockenheim	Germany	49.3278	8.56583	103
12	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.2486	264
13	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667	4
14	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139	401

78 rows

```
display(dbutils.fs.ls("abfss://demo@formula1dl1204.dfs.core.windows.net"))
```

Table

	path	name	size	modificationTime
1	abfss://demo@formula1dl1204.dfs.core.windows.net/circuits.c...	circuits.csv	10044	1721470200000

1 row

Table

	_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7
1	circuitId	circuitRef	name	location	country	lat	lng	alt
2	1	albert_park	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968	10
3	2	sepang	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738	18
4	3	bahrain	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106	7
5	4	catalunya	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111	109
6	5	istanbul	Istanbul Park	Istanbul	Turkey	40.9517	29.405	130
7	6	monaco	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056	7
8	7	villeneuve	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228	13
9	8	magny_cours	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361	228
10	9	silverstone	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694	153
11	10	hockenheimri...	Hockenheimring	Hockenheim	Germany	49.3278	8.56583	103
12	11	hungaroring	Hungaroring	Budapest	Hungary	47.5789	19.2486	264
13	12	valencia	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667	4
14	13	spa	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139	401

78 rows

Objectives

1. Spark SQL Dcumentation
2. Create database demo
3. Data tab in the UI
4. SHOW command
5. DESCRIBE command
6. Find the current database

```
create database IF not exists demo;
```

OK

```
SHOW DATABASES;
```

Table

	databaseName
1	default
2	demo

2 rows

```
DESC DATABASE demo;
```

Table

	database_description_item	database_description_value
1	Catalog Name	spark_catalog
2	Namespace Name	demo
3	Comment	
4	Location	dbfs:/user/hive/warehouse/demo.db
5	Owner	root

5 rows

```
DESC DATABASE EXTENDED demo;
```

Table

	database_description_item	database_description_value

1	Catalog Name	spark_catalog
2	Namespace Name	demo
3	Comment	
4	Location	dbfs:/user/hive/warehouse/demo.db
5	Owner	root
6	Properties	

6 rows

get current database name

```
SELECT CURRENT_DATABASE();
```

Table

	current_schema()
1	default

1 row

get tables present in nhe current database

```
show tables;
```

Query returned no results

Change database as required

```
USE demo;
```

OK

```
SELECT CURRENT_DATABASE();
```

Table

	current_schema()
1	demo

1 row

Create Managed Table

```
%run "../includes/configuration"
```

```
%python
race_results_df = spark.read.parquet(f"{presentation_folder_path}/race_results")
```

```
%python
race_results_df.write.mode("overwrite").format("parquet").saveAsTable("demo.race_results_python")
```

```
SHOW TABLES;
```

Table

	database	tableName	isTemporary
1	demo	race_results_python	false
2	demo	race_results_sql	false

2 rows

```
DESC  race_results_python
```

Table

	col_name	data_type	comment
1	race_year	int	null
2	race_name	string	null
3	race_date	timestamp	null
4	circuit_location	string	null
5	driver_name	string	null
6	driver_nationality	string	null
7	team	string	null
8	grid	int	null
9	fastest_lap	int	null
10	race_time	string	null
11	points	float	null
12	position	int	null
13	created_date	timestamp	null

13 rows

```
DESC EXTENDED race_results_python
```

Table

	col_name	data_type	comment
1	race_year	int	null

2	race_name	string	null
3	race_date	timestamp	null
4	circuit_location	string	null
5	driver_name	string	null
6	driver_nationality	string	null
7	team	string	null
8	grid	int	null
9	fastest_lap	int	null
10	race_time	string	null
11	points	float	null
12	position	int	null
13	created_date	timestamp	null
14			
15	# Detailed Table Information		

28 rows

```
SELECT *
FROM demo.race_results_python
where race_year = 2020;
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Lewis Hamilton	British
2	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Max Verstappen	Dutch
3	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Alexander Albon	Thai
4	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Lando Norris	British
5	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Carlos Sainz	Spanish
6	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Pierre Gasly	French
7	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Daniel Ricciardo	Australian
8	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Valtteri Bottas	Finnish
9	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Esteban Ocon	French
10	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Charles Leclerc	Monegasque
11	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Daniil Kvyat	Russian
12	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	George Russell	British
13	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Sebastian Vettel	German
14	2020	Bahrain Grand Prix	2020-11-29T14:10:00.000+00:...	Sakhir	Nicholas Latifi	Canadian

340 rows

```
CREATE or replace TABLE demo.race_results_sql
AS
SELECT *
FROM demo.race_results_python
where race_year = 2020;
```

Query returned no results

```
DESC EXTENDED demo.race_results_sql;
```

Table

	col_name	data_type

1	race_year	int
2	race_name	string
3	race_date	timestamp
4	circuit_location	string
5	driver_name	string
6	driver_nationality	string
7	team	string
8	grid	int
9	fastest_lap	int
10	race_time	string
11	points	float
12	position	int
13	created_date	timestamp
14		

32 rows

External Table

Objective

1. Create External Table using Python
2. Create External Table using SQL
3. Effect of dropping an External Table

using python

```
%python
race_results_df.write.mode("overwrite").format("parquet").option("path", f"{presentation_folder_path}/race_results_ext_py").saveAsT
```

```
desc extended demo.race_results_ext_py;
```

Table

	col_name	data_type	comment
1	race_year	int	null
2	race_name	string	null
3	race_date	timestamp	null
4	circuit_location	string	null
5	driver_name	string	null
6	driver_nationality	string	null
7	team	string	null
8	grid	int	null
9	fastest_lap	int	null
10	race_time	string	null
11	points	float	null
12	position	int	null
13	created_date	timestamp	null
14			
15	# Detailed Table Information		

28 rows

Creating External Table using SQL

```
CREATE TABLE demo.race_results_ext_sql
(
race_year INT,
race_name STRING,
race_date TIMESTAMP,
circuit_location STRING,
driver_name STRING,
driver_nationality STRING,
team STRING,
grid INT,
fastest_lap INT,
race_time STRING,
points FLOAT,
position INT,
created_date TIMESTAMP
)
USING parquet
LOCATION "/mnt/formula1dl1204/presentation/race_results_ext_sql";
```

OK

```
drop table demo.race_results_ext_sql;
```

OK

```
show tables in demo;
```

Table

	database	tableName	isTemporary
1	demo	race_results_ext_py	false
2	demo	race_results_ext_sql	false
3	demo	race_results_python	false
4	demo	race_results_sql	false

4 rows

```
insert into demo.race_results_ext_sql
select * from demo.race_results_ext_py where race_year = 2020;
```

OK

```
select * from demo.race_results_ext_sql;
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Max Verstappen	Dutch
2	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Valtteri Bottas	Finnish
3	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Lewis Hamilton	British
4	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Alexander Albon	Thai

5	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Lando Norris	British
6	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Carlos Sainz	Spanish
7	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Daniel Ricciardo	Australian
8	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Pierre Gasly	French
9	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Esteban Ocon	French
10	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Lance Stroll	Canadian
11	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Daniil Kvyat	Russian
12	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Kimi Räikkönen	Finnish
13	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Charles Leclerc	Monegasque
14	2020	Abu Dhabi Grand P...	2020-12-13T13:10:00.000+00:...	Abu Dhabi	Sebastian Vettel	German

340 rows

```
select * from demo.race_results_ext_py;
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Jenson Button	British
2	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Rubens Barrichello	Brazilian
3	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Mark Webber	Australian
4	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Sebastian Vettel	German
5	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Fernando Alonso	Spanish
6	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Felipe Massa	Brazilian
7	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nick Heidfeld	German
8	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nico Rosberg	German
9	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Lewis Hamilton	British
10	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Timo Glock	German
11	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Robert Kubica	Polish
12	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nelson Piquet Jr.	Brazilian
13	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Kazuki Nakajima	Japanese
14	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Giancarlo Fisichella	Italian

10,000+ rows | Truncated data due to row limit

Views

Objective

1. Create Temp View
2. Create Global Temp View
3. Create Permamnent View

```
create or replace temporary view v_race_results
as
select * from demo.race_results_python;
```

OK

```
create or replace global temporary view gv_race_results
as
select * from demo.race_results_python;
```

OK

```
select * from global_temp.gv_race_results;
```

Table

	race_year	race_name	race_date	circuit_location	driver_name	driver_nationality
1	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Jenson Button	British
2	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Rubens Barrichello	Brazilian
3	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Mark Webber	Australian
4	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Sebastian Vettel	German
5	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Fernando Alonso	Spanish
6	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Felipe Massa	Brazilian
7	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nick Heidfeld	German
8	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nico Rosberg	German
9	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Lewis Hamilton	British
10	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Timo Glock	German
11	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Robert Kubica	Polish
12	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Nelson Piquet Jr.	Brazilian
13	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Kazuki Nakajima	Japanese
14	2009	Spanish Grand Prix	2009-05-10T12:00:00.000+00:...	Montmeló	Giancarlo Fisichella	Italian

10,000+ rows | Truncated data due to row limit

```
create or replace view demo.pv_race_results
as
select * from demo.race_results_python;
```

OK

```
show tables;
```

Table

	database	tableName	isTemporary
1	demo	pv_race_results	false
2	demo	race_results_ext_py	false
3	demo	race_results_ext_sql	false
4	demo	race_results_python	false
5	demo	race_results_sql	false
6		v_race_results	true

6 rows

Ingest pit_stops.json file

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

Step 1 - Read the JSON file using the spark dataframe reader API

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType
```

```
lap_times_schema = StructType(fields=[StructField("raceId", IntegerType(), False),
                                      StructField("driverId", IntegerType(), True),
                                      StructField("lap", IntegerType(), True),
                                      StructField("position", IntegerType(), True),
                                      StructField("time", StringType(), True),
                                      StructField("milliseconds", IntegerType(), True)
                                     ])
```

```
lap_times_df = spark.read \
.schema(lap_times_schema) \
.csv(f"{raw_folder_path}/{v_file_date}/lap_times")
```

```
```

```
display(lap_times_df)
```

Table

	raceId	driverId	lap	position	time	milliseconds
112	1053	1	49	5	1:19.154	79154
113	1053	1	50	4	1:17.858	77858
114	1053	1	51	4	1:18.512	78512
115	1053	1	52	4	1:19.001	79001
116	1053	1	53	4	1:18.805	78805
117	1053	1	54	4	1:19.206	79206
118	1053	1	55	3	1:17.950	77950
119	1053	1	56	3	1:18.505	78505
120	1053	1	57	3	1:18.622	78622
121	1053	1	58	3	1:18.957	78957
122	1053	1	59	3	1:18.534	78534
123	1053	1	60	2	1:16.702	76702
124	1053	1	61	2	1:18.824	78824
125	1053	1	62	2	1:18.787	78787
126	1053	1	63	2	1:18.038	78038
127	1053	844	1	3	1:47.198	107198

1,124 rows

Step 2 - Rename columns and add new columns

- Rename driverId and raceId
- Add ingestion_date with current timestamp

```
lap_times_with_ingestion_date_df = add_ingestion_date(lap_times_df)
```

```
from pyspark.sql.functions import current_timestamp, lit
```

```
final_df = lap_times_df.withColumnRenamed("driverId", "driver_id") \
.withColumnRenamed("raceId", "race_id") \
.withColumn("ingestion_date", current_timestamp()) \
.withColumn("data_source", lit(v_data_source)) \
.withColumn("file_date", lit(v_file_date))
```

```
display(final_df)
```

Table

	race_id	driver_id	lap	position	time	milliseconds	ingestion_date
1	1053	830	1	1	1:38.603	98603	2024-07-24T13:51:50.109+00:...
2	1053	830	2	1	2:29.163	149163	2024-07-24T13:51:50.109+00:...
3	1053	830	3	1	2:23.247	143247	2024-07-24T13:51:50.109+00:...
4	1053	830	4	1	2:20.332	140332	2024-07-24T13:51:50.109+00:...
5	1053	830	5	1	2:25.691	145691	2024-07-24T13:51:50.109+00:...
6	1053	830	6	1	2:20.804	140804	2024-07-24T13:51:50.109+00:...
7	1053	830	7	1	1:36.303	96303	2024-07-24T13:51:50.109+00:...
8	1053	830	8	1	1:32.925	92925	2024-07-24T13:51:50.109+00:...
9	1053	830	9	1	1:30.953	90953	2024-07-24T13:51:50.109+00:...
10	1053	830	10	1	1:30.130	90130	2024-07-24T13:51:50.109+00:...

12	1053	830	12	1	1:28.814	88814	2024-07-24T13:51:50.109+00:...
13	1053	830	13	1	1:28.119	88119	2024-07-24T13:51:50.109+00:...
14	1053	830	14	1	1:27.740	87740	2024-07-24T13:51:50.109+00:...

1,124 rows

Step 3 - Write to output to processed container in parquet format

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.lap_times")
```

```
# overwrite_partition(final_df, 'f1_processed', 'lap_times', 'race_id')
```

```
merge_condition = "tgt.race_id = src.race_id AND tgt.driver_id = src.driver_id AND tgt.lap = src.lap AND tgt.race_id = src.race_id"
merge_delta_data(final_df, 'f1_processed', 'lap_times', processed_folder_path, merge_condition, 'race_id')
```

```
# display(spark.read.parquet(f"{processed_folder_path}/lap_times"))
```

```
%sql
select * from f1_processed.lap_times;
```

Table

	race_id	driver_id	lap	position	time	milliseconds	ingestion_date	
1	847	20	1	1	2:18.174	138174	2024-07-24T13:50:26.497+00:...	
2	847	20	2	1	2:06.919	126919	2024-07-24T13:50:26.497+00:...	
3	847	20	3	1	2:05.303	125303	2024-07-24T13:50:26.497+00:...	
4	847	20	4	1	2:05.715	125715	2024-07-24T13:50:26.497+00:...	
5	847	20	5	1	1:36.175	96175	2024-07-24T13:50:26.497+00:...	
6	847	20	6	1	1:34.827	94827	2024-07-24T13:50:26.497+00:...	
7	847	20	7	1	1:35.452	95452	2024-07-24T13:50:26.497+00:...	
8	847	20	8	1	1:41.724	101724	2024-07-24T13:50:26.497+00:...	
9	847	20	9	1	2:13.172	133172	2024-07-24T13:50:26.497+00:...	
10	847	20	10	1	2:08.183	128183	2024-07-24T13:50:26.497+00:...	
11	847	20	11	1	2:07.373	127373	2024-07-24T13:50:26.497+00:...	
12	847	20	12	1	2:01.938	121938	2024-07-24T13:50:26.497+00:...	
13	847	20	13	1	1:32.888	92888	2024-07-24T13:50:26.497+00:...	
14	847	20	14	1	1:33.332	93332	2024-07-24T13:50:26.497+00:...	

10,000+ rows | Truncated data due to row limit

```
Success
```

databricks demo / 7.sql_basic_demo

(https://databricks.com)

```
show databases;
```

Table

	databaseName
1	default
2	demo
3	f1_presentation
4	f1_processed
5	f1_raw

5 rows

```
select current_database();
```

Table

	current_schema()
1	default

1 row

```
use f1_processed;
```

OK

```
show tables;
```

Table

	database	tableName	isTemporary
1	f1_processed	circuits	false
2	f1_processed	constructors	false
3	f1_processed	drivers	false
4	f1_processed	lap_times	false
5	f1_processed	pit_stops	false
6	f1_processed	qualifying	false
7	f1_processed	races	false
8	f1_processed	results	false

8 rows

```
select *
from f1_processed.drivers
limit 100;
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	2024-07

100 rows

```
select *, split(name, ' ')
from f1_processed.drivers;
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	2024-07

853 rows

```
select *, split(name, ' ')[0] forename, split(name, ' ')[1] surname
from f1_processed.drivers;
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07

2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	2024-07

853 rows

```
select *, date_format(dob, 'dd-MM-yyyy' )
from f1_processed.drivers;
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07
9	9	kubica	88	KUB	Robert Kubica	1984-12-07	Polish	2024-07
10	10	glock	null	GLO	Timo Glock	1982-03-18	German	2024-07
11	11	sato	null	SAT	Takuma Sato	1977-01-28	Japanese	2024-07
12	12	piquet_jr	null	PIQ	Nelson Piquet Jr.	1985-07-25	Brazilian	2024-07
13	13	massa	19	MAS	Felipe Massa	1981-04-25	Brazilian	2024-07
14	14	coulthard	null	COU	David Coulthard	1971-03-27	British	2024-07

853 rows

```
select *, date_add(dob, 1 )
from f1_processed.drivers;
```

Table

	driver_id	driver_ref	number	code	name	dob	nationality	inge
1	1	hamilton	44	HAM	Lewis Hamilton	1985-01-07	British	2024-07
2	2	heidfeld	null	HEI	Nick Heidfeld	1977-05-10	German	2024-07
3	3	rosberg	6	ROS	Nico Rosberg	1985-06-27	German	2024-07
4	4	alonso	14	ALO	Fernando Alonso	1981-07-29	Spanish	2024-07
5	5	kovalainen	null	KOV	Heikki Kovalainen	1981-10-19	Finnish	2024-07
6	6	nakajima	null	NAK	Kazuki Nakajima	1985-01-11	Japanese	2024-07
7	7	bourdais	null	BOU	Sébastien Bourdais	1979-02-28	French	2024-07
8	8	raikkonen	7	RAI	Kimi Räikkönen	1979-10-17	Finnish	2024-07

Ingest qualifying json files

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

Step 1 - Read the JSON file using the spark dataframe reader API

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType
```

```
qualifying_schema = StructType(fields=[StructField("qualifyId", IntegerType(), False),
                                       StructField("raceId", IntegerType(), True),
                                       StructField("driverId", IntegerType(), True),
                                       StructField("constructorId", IntegerType(), True),
                                       StructField("number", IntegerType(), True),
                                       StructField("position", IntegerType(), True),
                                       StructField("q1", StringType(), True),
                                       StructField("q2", StringType(), True),
                                       StructField("q3", StringType(), True),
                                      ])
```

```
qualifying_df = spark.read \
.schema(qualifying_schema) \
.option("multiLine", True) \
.json(f"{raw_folder_path}/{v_file_date}/qualifying")
```

Step 2 - Rename columns and add new columns

- Rename qualifyingId, driverId, constructorId and raceId
- Add ingestion_date with current timestamp

```
qualifying_with_ingestion_date_df = add_ingestion_date(qualifying_df)
```

```
from pyspark.sql.functions import current_timestamp, lit
```

```
final_df = qualifying_df.withColumnRenamed("qualifyId", "qualify_id") \
.withColumnRenamed("driverId", "driver_id") \
.withColumnRenamed("raceId", "race_id") \
.withColumnRenamed("constructorId", "constructor_id") \
.withColumn("ingestion_date", current_timestamp()) \
.withColumn("data_source", lit(v_data_source)) \
.withColumn("file_date", lit(v_file_date))
```

Step 3 - Write to output to processed container in parquet format

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.qualifying")
```

```
# overwrite_partition(final_df, 'f1_processed', 'qualifying', 'race_id')
```

```
merge_condition = "tgt.qualify_id = src.qualify_id AND tgt.race_id = src.race_id"
merge_delta_data(final_df, 'f1_processed', 'qualifying', processed_folder_path, merge_condition, 'race_id')
```

```
# display(spark.read.parquet(f"{processed_folder_path}/qualifying"))
```

```
%sql
select * from f1_processed.qualifying;
```

Table

	qualify_id	race_id	driver_id	constructor_id	number	position	q1	q2
1	3374	339	17	9	6	1	1:51.886	1:48.210
2	3375	339	3	131	4	2	1:52.560	1:47.417
3	3376	339	20	9	5	3	1:47.632	1:46.828
4	3377	339	16	10	14	4	1:49.479	1:47.085
5	3378	339	807	3	10	5	1:49.664	1:47.346
6	3379	339	9	4	11	6	1:46.283	1:46.951
7	3380	339	22	3	9	7	1:50.301	1:48.371
8	3381	339	30	131	3	8	1:52.239	1:48.400
9	3382	339	155	15	23	9	1:48.467	1:47.792
10	3383	339	24	10	15	10	1:49.922	1:48.238
11	3384	339	808	4	12	11	1:47.952	1:48.760
12	3385	339	37	15	22	12	1:47.153	1:48.771
13	3386	339	67	5	16	13	1:48.945	1:49.207
14	3387	339	153	5	17	14	1:48.655	1:49.464

```
-- Databricks notebook source
CREATE DATABASE IF NOT EXISTS f1_processed
LOCATION "/mnt/formula1dl1204/processed"
```

OK

```
DESC DATABASE f1_processed;
```

Table

	database_description_item	database_description_value
1	Catalog Name	spark_catalog
2	Namespace Name	f1_processed
3	Comment	
4	Location	dbfs:/mnt/formula1dl1204/processed
5	Owner	root

5 rows

```
desc extended f1_processed.constructors;
```

Table

	col_name	data_type	comment
1	constructor_id	int	null
2	constructor_ref	string	null
3	name	string	null
4	nationality	string	null
5	data_source	string	null
6	file_date	string	null
7	ingestion_date	timestamp	null
8			
9	# Detailed Table Information		
10	Catalog	spark_catalog	
11	Database	f1_processed	
12	Table	constructors	

databricks 10.delta_lake_demo

(https://databricks.com)

1. Write data to delta lake (managed table)
2. Write data to delta lake (external table)
3. Read data from delta lake (Table)
4. Read data from delta lake (File)

2

```
%sql
CREATE DATABASE IF NOT EXISTS f1_demo
LOCATION '/mnt/formula1dl1204/demo'
```

OK

3

```
results_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/results.json")
```

4

```
results_df.write.format("delta").mode("overwrite").saveAsTable("f1_demo.results_managed")
```

5

```
%sql
SELECT * FROM f1_demo.results_managed;
```

Table

	constructorId	driverId	fastestLap	fastestLapSpeed	fastestLapTime	grid	laps
1	131	1	44	207.235	1:34.015	2	56
2	9	830	41	208.984	1:33.228	1	56
3	131	822	56	211.566	1:32.090	3	56
4	1	846	38	206.398	1:34.396	7	56
5	9	815	44	207.334	1:33.970	0	56
6	6	844	39	205.112	1:34.988	4	56
7	1	817	36	205.233	1:34.932	6	56
8	6	832	48	206.151	1:34.509	8	56
9	213	852	38	205.603	1:34.761	13	56
10	117	840	31	205.378	1:34.865	10	56
11	51	8	45	204.672	1:35.192	14	56
12	51	841	32	204.823	1:35.122	12	55
13	214	839	33	204.548	1:35.250	16	55
14	3	847	40	205.008	1:35.036	15	55

20 rows

6

```
results_df.write.format("delta").mode("overwrite").save("/mnt/formula1dl1204/demo/results_external")
```

7

```
%sql
CREATE TABLE f1_demo.results_external
USING DELTA
LOCATION '/mnt/formula1dl1204/demo/results_external'
```

OK

8

```
%sql
SELECT * FROM f1_demo.results_external
```

Table

	constructorId	driverId	fastestLap	fastestLapSpeed	fastestLapTime	grid	laps
1	131	1	44	207.235	1:34.015	2	56
2	9	830	41	208.984	1:33.228	1	56
3	131	822	56	211.566	1:32.090	3	56
4	1	846	38	206.398	1:34.396	7	56
5	9	815	44	207.334	1:33.970	0	56
6	6	844	39	205.112	1:34.988	4	56
7	1	817	36	205.233	1:34.932	6	56
8	6	832	48	206.151	1:34.509	8	56
9	213	852	38	205.603	1:34.761	13	56
10	117	840	31	205.378	1:34.865	10	56
11	51	8	45	204.672	1:35.192	14	56
12	51	841	32	204.823	1:35.122	12	55
13	214	839	33	204.548	1:35.250	16	55
14	3	847	40	205.008	1:35.036	15	55

20 rows

9

```
results_external_df = spark.read.format("delta").load("/mnt/formula1dl1204/demo/results_external")
```

10

```
display(results_external_df)
```

Table

	constructorId	driverId	fastestLap	fastestLapSpeed	fastestLapTime	grid	laps
1	131	1	44	207.235	1:34.015	2	56
2	9	830	41	208.984	1:33.228	1	56
3	131	822	56	211.566	1:32.090	3	56
4	1	846	38	206.398	1:34.396	7	56
5	9	815	44	207.334	1:33.970	0	56
6	6	844	39	205.112	1:34.988	4	56
7	1	817	36	205.233	1:34.932	6	56
8	6	832	48	206.151	1:34.509	8	56
9	213	852	38	205.603	1:34.761	13	56
10	117	840	31	205.378	1:34.865	10	56
11	51	8	45	204.672	1:35.192	14	56
12	51	841	32	204.823	1:35.122	12	55
13	214	839	33	204.548	1:35.250	16	55
14	3	847	40	205.008	1:35.036	15	55

20 rows

11

```
results_df.write.format("delta").mode("overwrite").partitionBy("constructorId").saveAsTable("f1_demo.results_partitioned")
```

12

```
%sql
SHOW PARTITIONS f1_demo.results_partitioned
```

Table

	constructorId
1	1
2	3
3	6
4	9
5	51
6	131
7	117
8	214
9	210
10	213

10 rows

1. Update Delta Table
2. Delete From Delta Table

14

```
%sql
SELECT * FROM f1_demo.results_managed;
```

15

```
%sql
UPDATE f1_demo.results_managed
  SET points = 11 - position
WHERE position <= 10
```

16

```
%sql
SELECT * FROM f1_demo.results_managed;
```

17

```
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/mnt/formula1dl1204/demo/results_managed")

deltaTable.update("position <= 10", { "points": "21 - position" } )
```

18

```
%sql
SELECT * FROM f1_demo.results_managed;
```

19

```
%sql
DELETE FROM f1_demo.results_managed
WHERE position > 10;
```

20

```
%sql
SELECT * FROM f1_demo.results_managed;
```

21

```
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/mnt/formula1dl1204/demo/results_managed")

deltaTable.delete("points = 0")
```

22

```
%sql
SELECT * FROM f1_demo.results_managed;
```

Upsert using merge

24

```
drivers_day1_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/drivers.json") \
.filter("driverId <= 10") \
.select("driverId", "dob", "name.forename", "name.surname")
```

25

```
display(drivers_day1_df)
```

26

```
drivers_day1_df.createOrReplaceTempView("drivers_day1")
```

27

```
from pyspark.sql.functions import upper

drivers_day2_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/drivers.json") \
.filter("driverId BETWEEN 6 AND 15") \
.select("driverId", "dob", upper("name.forename").alias("forename"), upper("name.surname").alias("surname"))
```

28

```
drivers_day2_df.createOrReplaceTempView("drivers_day2")
```

29

```
display(drivers_day2_df)
```

30

```
from pyspark.sql.functions import upper

drivers_day3_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/drivers.json") \
.filter("driverId BETWEEN 1 AND 5 OR driverId BETWEEN 16 AND 20") \
.select("driverId", "dob", upper("name.forename").alias("forename"), upper("name.surname").alias("surname"))
```

31

```
%sql
CREATE TABLE IF NOT EXISTS f1_demo.drivers_merge (
driverId INT,
dob DATE,
forename STRING,
surname STRING,
createdDate DATE,
updatedDate DATE
)
USING DELTA
```

Day1

33

```
%sql
MERGE INTO f1_demo.drivers_merge tgt
USING drivers_day1 upd
ON tgt.driverId = upd.driverId
WHEN MATCHED THEN
  UPDATE SET tgt.dob = upd.dob,
             tgt.forename = upd.forename,
             tgt.surname = upd.surname,
             tgt.updatedDate = current_timestamp
WHEN NOT MATCHED
  THEN INSERT (driverId, dob, forename,surname,createdDate ) VALUES (driverId, dob, forename,surname, current_timestamp)
```

34

```
%sql SELECT * FROM f1_demo.drivers_merge;
```

Day 2

36

```
%sql
MERGE INTO f1_demo.drivers_merge tgt
USING drivers_day2 upd
ON tgt.driverId = upd.driverId
WHEN MATCHED THEN
  UPDATE SET tgt.dob = upd.dob,
             tgt.forename = upd.forename,
             tgt.surname = upd.surname,
             tgt.updatedDate = current_timestamp
WHEN NOT MATCHED
  THEN INSERT (driverId, dob, forename,surname,createdDate ) VALUES (driverId, dob, forename,surname, current_timestamp)
```

37

```
%sql SELECT * FROM f1_demo.drivers_merge;
```

Day 3

39

```
from pyspark.sql.functions import current_timestamp
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/mnt/formula1dl1204/demo/drivers_merge")

deltaTable.alias("tgt").merge(
    drivers_day3_df.alias("upd"),
    "tgt.driverId = upd.driverId") \
  .whenMatchedUpdate(set = { "dob" : "upd.dob", "forename" : "upd.forename", "surname" : "upd.surname", "updatedDate": "current_tim
  .whenNotMatchedInsert(values =
    {
      "driverId": "upd.driverId",
      "dob": "upd.dob",
      "forename" : "upd.forename",
      "surname" : "upd.surname",
      "createdDate": "current_timestamp()"
    }
  ) \
  .execute()
```

40

```
%sql SELECT * FROM f1_demo.drivers_merge;
```

1. History & Versioning
2. Time Travel
3. Vaccum

42

```
%sql
DESC HISTORY f1_demo.drivers_merge
```

43

```
%sql
SELECT * FROM f1_demo.drivers_merge VERSION AS OF 2;
```

44

```
%sql
SELECT * FROM f1_demo.drivers_merge TIMESTAMP AS OF '2021-06-23T15:40:33.000+0000';
```

45

```
df = spark.read.format("delta").option("timestampAsOf", '2021-06-23T15:40:33.000+0000').load("/mnt/formula1dl1204/demo/drivers_merg
```

46

```
display(df)
```

47

```
%sql
VACUUM f1_demo.drivers_merge
```

48

```
%sql
SELECT * FROM f1_demo.drivers_merge TIMESTAMP AS OF '2021-06-23T15:40:33.000+0000';
```

49

```
%sql
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM f1_demo.drivers_merge RETAIN 0 HOURS
```

50

```
%sql
SELECT * FROM f1_demo.drivers_merge TIMESTAMP AS OF '2021-06-23T15:40:33.000+0000';
```

51

```
%sql
SELECT * FROM f1_demo.drivers_merge
```

52

```
%sql
DESC HISTORY f1_demo.drivers_merge;
```

53

```
%sql
DELETE FROM f1_demo.drivers_merge WHERE driverId = 1;
```

54

```
%sql
SELECT * FROM f1_demo.drivers_merge VERSION AS OF 3;
```

55

```
%sql
MERGE INTO f1_demo.drivers_merge tgt
USING f1_demo.drivers_merge VERSION AS OF 3 src
   ON (tgt.driverId = src.driverId)
WHEN NOT MATCHED THEN
   INSERT *
```

56

```
%sql DESC HISTORY f1_demo.drivers_merge
```

57

```
%sql
SELECT * FROM f1_demo.drivers_merge
```

Transaction Logs

59

```
%sql
CREATE TABLE IF NOT EXISTS f1_demo.drivers_txn (
driverId INT,
dob DATE,
forename STRING,
surname STRING,
createdDate DATE,
updatedDate DATE
)
USING DELTA
```

60

```
%sql
DESC HISTORY f1_demo.drivers_txn
```

61

```
%sql
INSERT INTO f1_demo.drivers_txn
SELECT * FROM f1_demo.drivers_merge
WHERE driverId = 1;
```

62

```
%sql
DESC HISTORY f1_demo.drivers_txn
```

63

```
%sql
INSERT INTO f1_demo.drivers_txn
SELECT * FROM f1_demo.drivers_merge
WHERE driverId = 2;
```

64

```
%sql
DELETE FROM  f1_demo.drivers_txn
WHERE driverId = 1;
```

65

```
for driver_id in range(3, 20):
  spark.sql(f"""INSERT INTO f1_demo.drivers_txn
                SELECT * FROM f1_demo.drivers_merge
                WHERE driverId = {driver_id}""")
```

66

```
%sql
INSERT INTO f1_demo.drivers_txn
SELECT * FROM f1_demo.drivers_merge;
```

Convert Parquet to Delta

68

```
%sql
CREATE TABLE IF NOT EXISTS f1_demo.drivers_convert_to_delta (
driverId INT,
dob DATE,
forename STRING,
surname STRING,
createdDate DATE,
updatedDate DATE
)
USING PARQUET
```

69

```
%sql
INSERT INTO f1_demo.drivers_convert_to_delta
SELECT * FROM f1_demo.drivers_merge
```

70

```
%sql
CONVERT TO DELTA f1_demo.drivers_convert_to_delta
```

71

```
df = spark.table("f1_demo.drivers_convert_to_delta")
```

72

```
df.write.format("parquet").save("/mnt/formula1dl1204/demo/drivers_convert_to_delta_new")
```

73

```
%sql
CONVERT TO DELTA parquet.`/mnt/formula1dl1204/demo/drivers_convert_to_delta_new`
```

74

1

```
# Databricks notebook source
from pyspark.sql.functions import current_timestamp
def add_ingestion_date(input_df):
  output_df = input_df.withColumn("ingestion_date", current_timestamp())
  return output_df
```

2

```
def re_arrange_partition_column(input_df, partition_column):
  column_list = []
  for column_name in input_df.schema.names:
    if column_name != partition_column:
      column_list.append(column_name)
  column_list.append(partition_column)
  output_df = input_df.select(column_list)
  return output_df
```

3

```
def overwrite_partition(input_df, db_name, table_name, partition_column):
  output_df = re_arrange_partition_column(input_df, partition_column)
  spark.conf.set("spark.sql.sources.partitionOverwriteMode","dynamic")
  if (spark._jsparkSession.catalog().tableExists(f"{db_name}.{table_name}")):
    output_df.write.mode("overwrite").insertInto(f"{db_name}.{table_name}")
  else:
    output_df.write.mode("overwrite").partitionBy(partition_column).format("parquet").saveAsTable(f"{db_name}.{table_name}")
```

4

```
def df_column_to_list(input_df, column_name):
  df_row_list = input_df.select(column_name) \
                        .distinct() \
                        .collect()

  column_value_list = [row[column_name] for row in df_row_list]
  return column_value_list
```

5

```
def merge_delta_data(input_df, db_name, table_name, folder_path, merge_condition, partition_column):
  spark.conf.set("spark.databricks.optimizer.dynamicPartitionPruning","true")

  from delta.tables import DeltaTable
  if (spark._jsparkSession.catalog().tableExists(f"{db_name}.{table_name}")):
    deltaTable = DeltaTable.forPath(spark, f"{folder_path}/{table_name}")
    deltaTable.alias("tgt").merge(
        input_df.alias("src"),
        merge_condition) \
      .whenMatchedUpdateAll()\
      .whenNotMatchedInsertAll()\
      .execute()
  else:
    input_df.write.mode("overwrite").partitionBy(partition_column).format("delta").saveAsTable(f"{db_name}.{table_name}")
```

6

```
# Databricks notebook source
raw_folder_path = "/mnt/formula1dl1204/raw"
processed_folder_path = "/mnt/formula1dl1204/processed"
presentation_folder_path = "/mnt/formula1dl1204/presentation"
```

```
# COMMAND ----------

#raw_folder_path = 'abfss://raw@formula1dl.dfs.core.windows.net'
#processed_folder_path = 'abfss://processed@formula1dl.dfs.core.windows.net'
#presentation_folder_path = 'abfss://presentation@formula1dl.dfs.core.windows.net'
```

databricks mount_adls_storage

(https://databricks.com)

1

```
storage_account_name = "formula1dl1204"
client_id            = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-id')
tenant_id            = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-tenant-id')
client_secret        = dbutils.secrets.get(scope = 'formula1-scope', key = 'formula1-app-client-secret')
```

2

```
configs = {"fs.azure.account.auth.type": "OAuth",
          "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
          "fs.azure.account.oauth2.client.id": client_id,
          "fs.azure.account.oauth2.client.secret": client_secret,
          "fs.azure.account.oauth2.client.endpoint": f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"}
```

3

```
def mount_adls(container_name):
  dbutils.fs.mount(
    source = f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/",
    mount_point = f"/mnt/{storage_account_name}/{container_name}",
    extra_configs = configs)
```

4

```
1  mount_adls("raw")
```

```
> ExecutionError: An error occurred while calling o416.mount.
: java.rmi.RemoteException: java.lang.IllegalArgumentException: requirement failed: Directory already mounted: /mnt/formula1dl120
4/raw; nested exception is: …
File <command-4320014699931724>, line 1
----> 1 mount_adls("raw")

File /databricks/python_shell/dbruntime/dbutils.py:378, in DBUtils.FSHandler.prettify_exception_message.<locals>.f_with_exception_
handling(*args, **kwargs)
    376 exc.__context__ = None
    377 exc.__cause__ = None
--> 378 raise exc
```

5

```
1  mount_adls("processed")
```

```
> ExecutionError: An error occurred while calling o416.mount.
: java.rmi.RemoteException: java.lang.IllegalArgumentException: requirement failed: Directory already mounted: /mnt/formula1dl120
4/processed; nested exception is: …
File <command-4320014699931725>, line 1
----> 1 mount_adls("processed")

File /databricks/python_shell/dbruntime/dbutils.py:378, in DBUtils.FSHandler.prettify_exception_message.<locals>.f_with_exception_
handling(*args, **kwargs)
    376 exc.__context__ = None
    377 exc.__cause__ = None
--> 378 raise exc
```

6

```
1  mount_adls("presentation")
```

```
> ExecutionError: An error occurred while calling o416.mount.
: java.rmi.RemoteException: java.lang.IllegalArgumentException: requirement failed: Directory already mounted: /mnt/formula1dl120
4/presentation; nested exception is: …
```

```
File <command-4320014699931726>, line 1
----> 1 mount_adls("presentation")
File /databricks/python_shell/dbruntime/dbutils.py:378, in DBUtils.FSHandler.prettify_exception_message.<locals>.f_with_exception_handling(*args, **kwargs)
    376 exc.__context__ = None
    377 exc.__cause__ = None
--> 378 raise exc
```

7

```
mount_adls("demo")
```

```
ExecutionError: An error occurred while calling o416.mount.
: java.rmi.RemoteException: java.lang.IllegalArgumentException: requirement failed: Directory already mounted: /mnt/formula1dl1204/demo; nested exception is: …
File <command-4320014699931727>, line 1
----> 1 mount_adls("demo")
File /databricks/python_shell/dbruntime/dbutils.py:378, in DBUtils.FSHandler.prettify_exception_message.<locals>.f_with_exception_handling(*args, **kwargs)
    376 exc.__context__ = None
    377 exc.__cause__ = None
--> 378 raise exc
```

8

```
dbutils.fs.unmount("/mnt/formula1dl1204/demo")
```

```
/mnt/formula1dl1204/demo has been unmounted.

True
```

9

```
dbutils.fs.ls("/mnt/formula1dl1204/raw")
```

```
[FileInfo(path='dbfs:/mnt/formula1dl1204/raw/2021-03-21/', name='2021-03-21/', size=0, modificationTime=1721668310000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/raw/2021-03-28/', name='2021-03-28/', size=0, modificationTime=1721668310000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/raw/2021-04-18/', name='2021-04-18/', size=0, modificationTime=1721668310000)]
```

10

```
dbutils.fs.ls("/mnt/formula1dl1204/processed")
```

```
[FileInfo(path='dbfs:/mnt/formula1dl1204/processed/circuits/', name='circuits/', size=0, modificationTime=1721816401000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/processed/constructors/', name='constructors/', size=0, modificationTime=1721818389000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/processed/lap_times/', name='lap_times/', size=0, modificationTime=1721818108000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/processed/pit_stops/', name='pit_stops/', size=0, modificationTime=1721815790000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/processed/qualifying/', name='qualifying/', size=0, modificationTime=1721818131000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/processed/races/', name='races/', size=0, modificationTime=1721816423000),
 FileInfo(path='dbfs:/mnt/formula1dl1204/processed/results/', name='results/', size=0, modificationTime=1721672312000)]
```

11

```
dbutils.fs.ls("/mnt/formula1dl1204/presentation")
```

```
[]
```

12

```
dbutils.fs.ls("/mnt/formula1dl1204/demo")
```

```
[FileInfo(path='dbfs:/mnt/formula1dl1204/demo/circuits.csv', name='circuits.csv', size=10044, modificationTime=1721470200000)]
```

13